Benchmarking and Detecting Explanation Inconsistencies in LLM-based Vulnerability Repair

Lam Nguyen Tung, Aldeida Aleti, Xiaoning Du, Neelofar Neelofar, Matias Martinez, Justyna Petke, Federica Sarro

***Abstract*—LLMs have shown remarkable progress in automated vulnerability repair (AVR). Current AVR systems primarily focus on generating patches that satisfy conventional correctness criteria, such as passing test suites, preserving functionality, and eliminating vulnerabilities. However, these criteria do not necessarily reflect LLMs' true understanding. LLMs can rely on spurious correlations in training data or on memorization. They can generate correct patches with wrong explanations due to misunderstanding the semantic root cause of vulnerabilities. Existing AVR evaluations may therefore overestimate model capability, as repair success may reflect superficial pattern recall rather than genuine reasoning. This leads to *untrustworthy* predictions. Consequently, when AVR systems are deployed with unseen real-world vulnerabilities, their performance often deteriorates significantly. Hence, evaluating and detecting untrustworthy predictions is crucial. This, however, remains underexplored.**

We address this gap with the first benchmark and automated detection approach for untrustworthy predictions in LLM-based AVR. Our benchmark, VFXplainBench, contains 11,210 predictions generated by multiple systems using both commercial and open-source LLM backends. We find that 17.2% of correct patch predictions are actually untrustworthy. To detect them, we propose SUSVF, an automated, reference-free method that identifies inconsistencies between the generated patch and its explanation, within the explanation's reasoning steps, and between the explanation and the vulnerable code. On VFXplainBench, SUSVF achieves 0.882 precision, 0.732 recall, and 0.768 MCC, substantially outperforming LLM-as-a-Judge. These results show that patch correctness alone is insufficient for evaluating AVR systems. They also highlight the importance of explanation-based untrustworthiness detection for safer deployment of LLMs.

***Index Terms*—Vulnerability Repair, Explanation, Trustworthy**

I. INTRODUCTION

Large language models (LLMs) have revolutionized automated vulnerability repair (AVR) [1], achieving state-of-the-art (SOTA) success rates [2], [3]. Most studies on AVR reliability have investigated if a generated patch is correct, an undecidable problem [4]. They often rely on tests, considering a patch correct if it passes a test suite. Test-based validation suffers from overfitting [5], [6], where patches pass an incomplete test suite, making them plausible but in fact incorrect. This has motivated extensive research on patch correctness assessment [7]. However, such research only evaluates *what* an AVR system generates, not *why* the system generates it.

This distinction matters because patch correctness does not necessarily reflect the system's true understanding and may overlook issues in its decision-making. LLMs have been shown to be untruthful [8], misunderstand vulnerabilities [9], hallucinate [2], memorize fixes [10], or rely on spurious correlations [8]. As a result, they may generate correct patches spuriously. We refer to such cases as *untrustworthy* predictions[1]. Because their patches are correct, patch correctness assessment accepts them, hiding their untrustworthiness.

Untrustworthy predictions can be revealed by LLM explanations, e.g., chain-of-thought (CoT) intermediate reasoning steps [11]. These explanations have emerged as an important basis to understand LLMs' decision-making [12]–[14]. They explicitly illustrate in natural language (NL) the reasoning process by which LLMs generate patches, exposing the *why* behind patches. Hence, explanations allow us to audit repair rationale and diagnose untrustworthy predictions, in particular, where the patches are correct but with wrong explanations.

It is important to detect untrustworthy predictions [14], [15]. If left undetected, they inflate LLMs' perceived capability during evaluation, as they can generate correct patches by chance. When LLMs are deployed on unseen vulnerabilities, their performance often deteriorates [14], [16]. Hence, detecting untrustworthy predictions helps practitioners identify behavioral issues, better grasp the true AVR capability, and mitigate overtrust that may lead to unreliable patches. This detection allows us to test the trustworthiness of LLMs for AVR in addition to patch-correctness evaluation. Moreover, explanations are necessary to prepare relevant documents for review or maintenance. Untrustworthy predictions with wrong explanations can be misleading, so detecting them is needed.

Example. Figure 1 shows a C program affected by the CVE-2018-16421[2] vulnerability. The program copies `cipher_len-2` bytes from `plaintxt` to `out`, and may write outside `out`'s intended boundary, denoted by `*out_len`. The figure also includes a prediction of DeepSeek-Coder-33B-Instruct, which contains a patch and its explanation. The patch is correct, adding a boundary check to avoid `*out_len < cipher_len-2`. While the explanation seems logical, it proposes a fixing action different from the patch. This makes the prediction untrustworthy, likely because the model memorised the correct patch during training. The explanation reveals the LLM fails to capture the vulnerability's root cause and only spuriously addresses it [10]. This may leave similar vulnerabilities unpatched. Users also worry that overreliance on LLMs can degrade their programming skills [13]. They can make poor decisions about future changes or propagate flawed rationale to related components.

Our Goal. LLM-provided explanations can act as a useful

[1]We define a prediction an LLM's generation of a patch and its explanation
[2]https://nvd.nist.gov/vuln/detail/CVE-2018-16421

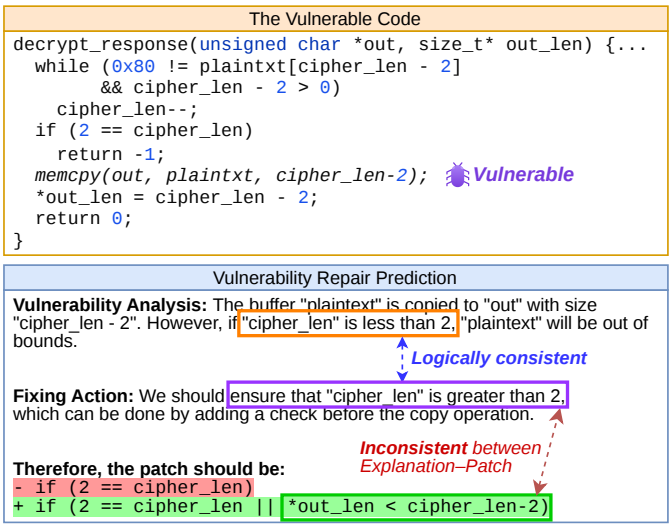


Fig. 1: A program with the CVE-2018-16421 vulnerability (top), where writes may occur outside`out`'s intended boundary, denoted by `*out_len`. The AVR prediction of DeepSeek-Coder-33B-Instruct (bottom) provides a patch and its explanation. Although the patch is correct and the explanation itself is logically consistent, the explanation proposes a fixing action inconsistent with the patch.

basis for filtering out untrustworthy predictions. However, reviewing them remains manual, which is time-consuming and unscalable. This motivates the need for a mechanism that can flag untrustworthy predictions automatically.

Key Insight. The explanation of a trustworthy prediction should be not only logically consistent (i.e., the proposed action can fix the analysed vulnerability) but also consistent with both the patch and the vulnerable code. The explanation, patch and vulnerable code are three essential artifacts of LLM-based AVR. This naturally motivates examining consistencies between them. Consistencies alone may not guarantee correctness and trustworthiness. However, inconsistencies mean the explanation states something misaligned with the patch, its own reasoning steps, or the vulnerable code. Here, inconsistencies serve as a strong *sufficient condition* for untrustworthiness, which enables the detection of untrustworthy predictions. Such detection is principled, not solely heuristic.

We believe it is theoretically infeasible to exhaustively identify all untrustworthy predictions, as it would require intelligence and knowledge similar to or exceeding that of AVR itself. Assessing inconsistencies provides a safe, lightweight and conservative detection without requiring additional references. Specifically, *if an explanation is not consistent, its corresponding patch prediction is untrustworthy.* This enables a reference-free oracle for LLM-based AVR that can detect untrustworthy predictions with minimal false alarms.

Approach. We develop SUSVF, the first automated approach that can **S**pot **u**ntrus**t**worthy LLM-based predictions for **V**ulnerability **F**ixes. SUSVF contains three stages that assess inconsistencies across ① the patch and the explanation, ② the explanation's reasoning steps, and ③ the explanation and the vulnerable code. While ③ can be assessed by rule-based and static code analyses, ① and ② can be evaluated by learning-based semantic analysis and natural language inference (NLI). Learning-based approaches may themselves raise trustworthiness concerns. Hence, we deliberately reduce their scope to tasks substantially simpler than AVR, such as semantic similarity measurement, short-document summarisation, and basic NLI, which can be reliably handled by current SOTA models. With that in mind, we carefully design criteria and apply techniques to enhance SUSVF's own trustworthiness.

In Stage ①, SUSVF generates an NL description of the code changes in the patch by summarising fine-grained differences identified by tools (e.g., Gumtree [17]). The explanation–patch inconsistency is estimated by semantic equivalence between the generated description and the explanation. In Stage ②, SUSVF frames the evaluation of the logical inconsistency between an explanation's reasoning steps as an NLI task. Unlike AVR, which requires extensive domain knowledge, we leverage LLMs' NLI capabilities by rigorously prompting them. In Stage ③, SUSVF assesses inconsistency between identifiers in the patch and the explanation by applying static code analysis. Finally, a prediction is labeled untrustworthy if inconsistency is observed in any stage.

To evaluate SUSVF and characterize the untrustworthiness of LLM-based AVR, we construct VFXplainBench. It is a manually-validated trustworthiness benchmark of 11,210 AVR predictions. VFXplainBench covers multiple SOTA AVR systems [2], [18]–[21] and vulnerability datasets [19], [22]–[24]. To our knowledge, no comparable benchmark exists.

We evaluate how well SUSVF detects untrustworthy predictions among correct patch predictions in VFXplainBench. SUSVF achieves 0.882 in Precision, 0.732 in Recall, and 0.768 in Matthews correlation coefficient (MCC). It also outperforms LLM-as-a-Judge [25] methods by up to 589% in Precision, 4,256% in Recall, and 4,367% in MCC. We summarise our contributions as follows:

- A trustworthiness benchmark, VFXplainBench, of 11,210 predictions made by multiple LLM-based AVR systems;
- A reference-free criterion for untrustworthy AVR predictions based on explanation inconsistencies;
- An approach for automated detection of untrustworthy AVR predictions, SUSVF, which in our experiments achieved 0.882 precision and 0.732 recall;
- An empirical validation of the hypothesis that inconsistency is a sufficient condition for untrustworthiness, based on human analysis of untrustworthy predictions and on the agreement between SUSVF and these labels;
- A comprehensive evaluation showing SUSVF's superior effectiveness over other approaches; and
- A replication package: **zenodo.org/records/18403393**.

II. Preliminary and Background

A. Definitions

1) Explanation: As LLMs are inherently black-box, several techniques have been explored to provide explanations for their predictions. Recently, CoT [11] and its variants [26], [27] have gained popularity. They explicitly outline sequences of intermediate reasoning steps that lead to a final answer,

offering interpretable windows into model behavior [11]. CoT prompting has been widely applied to SE tasks, including AVR, to enhance performance and provide explanations [2].

In this study, we define an *explanation* as CoT intermediate reasoning steps expressed in NL. They often describe vulnerability semantics based on relevant lines of code, analyse the root cause of the vulnerability, and propose a fixing action (bottom of Figure 1). These explanations differ fundamentally from post-patch explanations. A post-patch explanation is generated independently as a retrospective justification for an already-determined patch, rather than reflecting the LLM's reasoning. As a result, unlike CoT intermediate steps that arise during generation, post-patch explanations may not faithfully represent the reasoning that generated the patch.

2) Trustworthiness: In the context of LLMs, trustworthiness often refers to their intrinsic quality, a combination of robustness, security, transparency, fairness, and safety [13], [28]. We focus on trustworthiness, in particular, whether an LLM predicts correct patches with correct explanations rather than spurious ones. Hence, we adopt the definition of Kästner et al. [29] as shown in Definition 1.

Definition 1. A system is trustworthy to a stakeholder in a given context iff (a) the system works properly in the context and (b) the stakeholder would be justified to believe (a).

In AVR, patch correctness covers (a), showing that the program is correctly fixed. On the other hand, understanding a system thoroughly helps justify a reviewer's beliefs about how well it works. Explanations can serve as a means to gain this understanding, and the main evidence for (b). A wrong explanation cannot justify the reviewer's belief that the LLM works, even if the generated patch happens to be correct. As a result, a trustworthy AVR system predict correct patches, and explanations behind their predictions are also correct. We propose the definition of a trustworthy AVR prediction.

Definition 2. A trustworthy AVR prediction generates a correct patch and the explanation behind it is also correct.

A prediction may fall into one of three cases: (1) an incorrect patch, (2) a correct patch with a correct explanation, or (3) a correct patch but with a wrong explanation. In (1), investigating patch incorrectness becomes more important than trustworthiness testing. Patch correctness assessment has been widely studied [7] and is not our focus. Following prior work [14], [15] in trustworthiness testing, we examine only correct patch predictions and differentiate (2) represents trustworthy predictions from (3) indicates untrustworthy ones.

An untrustworthy prediction is not necessarily useless in practice. LLMs can make an overwhelming number of predictions, making it difficult to review them all. Hence, *trustworthiness* lets users narrow their options and focus their resources on those most likely to be deployed. Trustworthiness is also a more reliable metric than patch correctness to evaluate LLMs for AVR. Moreover, explanations are deliverables that users can use for deciding whether to accept generated patches and for downstream artifacts, e.g., commit messages or security advisories. Therefore, trustworthiness is a crucial property, and it is necessary to detect untrustworthy explanations.

We note that predictions with *partially incorrect* explanations are still untrustworthy. They may be even more dangerous, as their flaws are subtler and can mislead users more easily. Definition 2, the annotation process, and SUSVF are all designed to treat them as untrustworthy.

3) Inconsistency: Adapted from prior work on LLM hallucination [30], a correct explanation for a correct patch should remain logically coherent and well-aligned with both the patch and code context. We believe it is theoretically infeasible to exhaustively verify this, as it would require intelligence and knowledge similar to that of AVR itself. For that reason, we treat inconsistency as a *sufficient condition* for incorrect explanations, which is later leveraged in Sections III–IV.

Given three main artifacts in AVR: the vulnerable code, the patch, and the explanation, three types of inconsistencies can arise: inconsistency between the patch and the explanation, inconsistency in the explanation's reasoning steps, and inconsistency between the vulnerable code and the explanation. We do not consider the relationship between the vulnerable code and the patch, as we only focus on correct patch predictions.

Definition 3. An explanation is inconsistent if it exhibits any violation of alignment with respect to the predicted patch, its own reasoning steps, or the vulnerable code.

In this paper, inconsistency is a form of non-uniformity, distinct from superficial differences in how an explanation is expressed, such as paraphrasing or a different level of abstraction or granularity. Such differences do not by themselves make an explanation wrong. For instance, "ensure that the index is within the valid range" is a coarser description of changing > to >= in a bounds check, not an inconsistency with it. We apply this distinction in both the manual annotation in Section III-D and SUSVF's design in Section IV. We will describe how to quantify inconsistency in Section IV.

Inconsistency serves as a sufficient (but not necessary) condition for untrustworthiness. This enables high-precision detection of untrustworthy predictions, albeit potentially at the cost of recall. Notably, some untrustworthy predictions may produce consistent explanations. For example, Appendix C shows an untrustworthy prediction: its explanation is consistent, but the vulnerability's root cause is inadequate.

B. Application of Untrustworthiness Detection

Untrustworthiness, i.e., LLMs generate correct answers with wrong explanations, exists in various domains [10], [14], [15], including AVR [2]. LLMs often achieve high success rates on AVR benchmarks but are unlikely to perform as well in real-world deployment, especially on code beyond their training cutoff. This suggests that benchmark results may inflate LLMs' actual capabilities. Hence, it is crucial to assess LLM reasoning to detect untrustworthiness prior to deployment [14].

Prior studies [31], [32] indicate a strong preference among developers to remain involved in the repair process, rather than relying on fully automated suggestions. This is mainly due to trust issues. Developers need to correctly understand the rationale behind the suggestions to trust and implement

them effectively [2]. Hence, untrustworthy predictions may be disregarded. Traditionally, developers often detect them by manually reviewing LLMs' explanations accompanying the patches, which is not scalable [14].

Automating this detection can greatly advance LLMs' development and applications. It would streamline the review of predictions and enable automated trustworthiness testing within CI/CD pipelines. These activities are vital and closely intertwined with others in SE [33]. This is especially valuable in the iterative development of LLM-based systems where performance and trustworthiness must be evaluated and refined [34]. Beyond development, automated detection can monitor these systems in real-world environments, particularly in online testing or DevOps workflows. It can detect untrustworthiness in real time, foster greater human trust, and test real-world and corner-case inputs. Moreover, its outputs provide actionable feedback for improving LLM-based systems.

III. VFXplainBench: Trustworthiness Benchmark of LLM-based Vulnerability Fixes

As far as we know, no benchmark exists for the untrustworthiness of LLM-based AVR. Hence, we construct VFXplainBench by collecting the predictions of multiple SOTA LLM-based AVR systems and manually identifying untrustworthy ones. We investigate the first research question (RQ).

RQ1 [Benchmark]. *Are vulnerability fixes generated in a trustworthy manner?* Section V-A later introduces **RQ2–RQ5** for detecting untrustworthiness using VFXplainBench.

A. Model Selection

We select LLM-based AVR tools peer-reviewed and published within five years at top venues in SE, AI, and Security, including 13 conferences (ICSE, FSE, ASE, ISSTA, CCS, S&P, USENIX Security, NDSS, AAAI, IJCAI, ICML, NIPS, ICLR) and three journals (TOSEM, TSE, EMSE) [2]. We conduct a manual search to find relevant work using keywords *("LLM" OR "Large Language Model") AND ("Vulner*" OR "Secur*") AND ("Repair" OR "Fix" OR "Patch")* [2]. We focus specifically on tools that take vulnerable code as input and generate both a patch and an NL explanation. At the time of writing, few such tools exist, as most only generate patches, e.g., VulRepair [35], VulMaster [1], PatchAgent [36]. We also exclude San2Patch [37], which requires sanitizer logs as input. We thus choose the following tools.

- **Instruction Prompting** (IP) [2], [18] enables off-the-shelf LLMs to address AVR using CoT prompting. We emulate the multi-step vulnerability analysis of human security experts: gain an overview of the code, analyse critical subcomponents to pinpoint the vulnerability trigger, i.e., the condition under which it occurs, propose a solution, and create a patch [9], [38]. Each prompt includes the vulnerable function, the common weakness enumeration ids (CWE-IDs), and the vulnerability location [20].
- **AppPatch** [19] isolates a minimal yet sufficient code slice, where it identifies the likely root cause. AppPatch dynamically retrieves relevant exemplars from a pre-mined human-written database to guide the model's reasoning. It then applies these contexts and few-shot CoT prompting to produce patches and explanations.
- **Vul-R2** [21] is a fine-tuned reasoning LLM for AVR. It first constructs and filters reasoning data for supervised fine-tuning. Then, Vul-R2 applies curriculum-based reinforcement learning with verifiable rewards based on multiple-choice question answering and character-level matching.

TABLE I: Except for Vul-R2, we use the following LLMs as backends for Instruction Prompting and AppPatch

Model		Params	Type
GPT-4	gpt-4-0613	1.76T	Closed-source
Claude-Opus 4.1	claude-opus-4-1-20250805	-	
Qwen2.5-Coder-32B-Instruct	https://huggingface.co/Qwen/-Qwen2.5-Coder-32B-Instruct	32B	Open-source
CodeLlama-34B-Instruct	https://huggingface.co/codellama/CodeLlama-34b-Instruct-hf	34B	
DeepSeek-Coder-33B-Instruct	https://huggingface.co/deepseek-ai/deepseek-coder-33b-instruct	33B	

With IP and AppPatch, we use five SOTA LLMs as backends, covering both closed- and open-source models. For closed-source, we choose GPT-4 and Claude-Opus 4.1, leading models in SE [39]. For open-source, we consider CodeLlama, DeepSeek-Coder, and Qwen2.5-Coder, all successful in SE [2]. We use their "Instruct" variants, which are fine-tuned to follow instructions and adapt better to AVR, and select LLMs of similar size for a fair comparison. Table I lists all backend LLMs for IP and AppPatch, which we refer to as GPT-4, Claude-O 4.1, Qwen-Coder, CodeLlama, and DS-Coder.

B. Vulnerability Datasets

To collect predictions, we use vulnerability datasets from the tools' original evaluations, which were processed to avoid data leakage [19], [21]. These datasets are as follows.

- **ExtractFix** [22] and **ZeroDay** [19], [20]: ExtractFix is a widely-used C/C++ vulnerability benchmark, while ZeroDay is a new dataset that covers C/C++ vulnerabilities reported after 04/2024. In total, these datasets contain 319 vulnerabilities. They contain both single-hunk vulnerabilities and multi-hunk ones spanning multiple locations in one or multiple functions. Details are shown in Appendix B-3. We use these datasets to evaluate IP and AppPatch.
- **PrimeVul** (test set) [24] and **SVEN** [23]: PrimeVul is a large-scale real-world C/C++ dataset. It aggregates vulnerability-related commits from several established datasets, applies rigorous cleaning, and uses a temporally aware split into training, validation, and test sets. SVEN manually curates vulnerabilities from multiple C/C++ repositories, with 94% label accuracy confirmed by manual inspection [24]. While PrimeVul's training and validation sets are used to tune Vul-R2, its test set and SVEN, which contain 435 and 367 vulnerabilities, respectively, are used for evaluation. Both datasets contain only single-hunk and single-function multi-hunk vulnerabilities.

Overall, these datasets contain 1,121 vulnerabilities, covering 57 CWE-IDs.

TABLE II: **RQ1–VFXplainBench**. The trustworthiness benchmark of LLM-based AVR. IP and APPATCH are evaluated on ExtractFix [22] and ZeroDay [19], [20]. With five backend LLMs, these tools produce $319 \times 5 \times 2 = 3,190$ predictions on ExtractFix and ZeroDay. Vul-R2 is evaluated on PrimeVul's test set [24] and SVEN [23], fixing each vulnerability 10 times. This results in $435 \times 10 = 4,350$ predictions on PrimeVul's test set and $367 \times 10 = 3,670$ predictions on SVEN.

Data	Type	Vulnerabilities	Predictions	Correct Patches	Untrustworthy
ExtractFix [22], ZeroDay [19]	single-function, multi-function, single-hunk, multi-hunk	319	3,190	700	80 (11.4%)
PrimeVul's test set [24]	single-function, single-hunk, multi-hunk	435	4,350	532	98 (18.4%)
SVEN [23]	single-function, single-hunk, multi-hunk	367	3,670	446	111 (24.9%)
Total		1,121	11,210	1,678	289 (17.2%)

C. Collecting Predictions

We used the AVR tools in Section III-A to fix the vulnerabilities in Section III-B, collecting explanations and patches.

We replicated IP and APPATCH using five LLM backends shown in Table I with the same configurations, such as temperature, as in the original studies[3]. GPT-4 and Claude-O 4.1 were accessed remotely using OpenAI and Anthropic APIs, respectively. The open-source LLMs were deployed locally via HuggingFace APIs. We experimented on a Rocky Linux server with 24 cores of Intel(R) Xeon(R) Gold 6150 CPU @ 2.70GHz, 256GB RAM, and NVIDIA A100 GPU with 40GB memory. For each setting, we applied IP and APPATCH to repair 319 vulnerabilities in ExtractFix and ZeroDay. As a result, we collected $319 \times 5 = 1,595$ predictions each tool.

For Vul-R2, although the authors have not released the model, they provided the generated explanations and patches[4]. Vul-R2 attempts to repair each vulnerability in 10 runs. This resulted in $435 \times 10 = 4,350$ predictions on PrimeVul's test set and $367 \times 10 = 3,670$ predictions on SVEN. In total, we collected 11,210 predictions, as shown in Table II.

D. Trustworthiness Labeling

1) ***Patch correctness****:* This paper aims to determine if LLMs generate correct patches in a trustworthy manner. Therefore, we first identified correct patch predictions. Automated patch validation is infeasible. This is because the samples come from many projects, which makes compiling each one impractical, and most lack test cases. We thus determined each patch's correctness manually by comparing it with the developer patch. Following established practice [19], [21], a patch is labeled correct only if it exactly matches the developer patch, or is semantically equivalent to the developer patch according to the rules[5] in [40]. Other patches are labeled incorrect, *including partially correct patches.* An example is discussed in Appendix D.

Two authors, each with over five years of program repair experience, independently judged each generated patch's correctness against the developer patch, accepting both exact matches and semantic equivalence. In cases of disagreement, a third author, also with over five years of relevant experience, participated in the discussion to reach a consensus.

This procedure mitigates two known pitfalls of patch validation. First, because validation is manual rather than test-based, it avoids *overfitting*, where an incorrect patch passes an incomplete test suite [5], [6]. Second, it addresses the *undecidability* of patch validation [4] by safely and conservatively accepting only exact or semantically equivalent matches under the rules in [40]. With two independent annotators and a third resolving disagreements, patch validation becomes decidable and incorrect patches are unlikely to be labeled correct.

At the end, we identified 1,678 predictions with correct patches, as shown in Table II. As discussed in Definition 2, our study focuses on differentiating trustworthy and untrustworthy cases among correct patch predictions. Hence, we use these 1,678 predictions to evaluate SUSVF.

2) ***Explanation correctness****:* We then labeled each correct patch prediction as trustworthy if its explanation is correct. This labeling can be subjective, as it may involve human interpretation. To mitigate this, we applied the following annotation process. The annotators inspected each explanation against human-validated ground-truth references in National Vulnerability Database[6] (NVD): the developer patch, the vulnerability description and the message of the fixing commit. The annotators were explicitly instructed to judge whether the explanation is faithful to the NVD records rather than freely interpret correctness. Appendix D shows a screenshot of information provided to annotators for labeling. Any unfaithful information, even if only partially incorrect, makes an explanation incorrect. Differences in wording, abstraction, or granularity do not warrant an incorrect label if the explanation remained faithful to the NVD records. Consequently, for each prediction, the annotators gave a trustworthiness label, a confidence score on a five-point Likert scale, and a detailed rationale. Where ambiguity made the annotators unsure, they indicated this using the five-point confidence score. Disagreements and unsure cases were resolved through discussion with the third annotator.

Importantly, the annotators were not aware of any inconsistency types or SUSVF's detection rules. These types were derived after labeling, and SUSVF was designed after that.

At the end, among 1,678 correct patch predictions, 289 of them (17.2%) are deemed untrustworthy, as shown in Table II. The annotation process achieves a Cohen's Kappa value [41] of 0.84 for inter-reviewer agreement, indicating substantial consistency and reliability. Despite this, we acknowledge that

[3]https://zenodo.org/records/14741018

[4]https://github.com/Xin-Cheng-Wen/Vul-R2

[5]We applied only the rules relevant to C/C++

[6]https://nvd.nist.gov/

TABLE III: **RQ1–VFXplainBench**. Comparing the untrustworthiness between IP that uses zero-shot CoT prompting and APPATCH that applies few-shot CoT prompting. Each combination of AVR tools (IP, APPATCH) and LLM backends (GPT-4, Claude-O 4.1, Qwen-Coder, CodeLlama, DS-Coder) produces 319 predictions

Backend LLM	Predictions	IP (zero-shot)		APPATCH (few-shot)	
		Correct Patches	Untrustworthy	Correct Patches	Untrustworthy
GPT-4	319 per model	91	12 (13.2%)	92 $^{\uparrow 1}$	6 $^{\downarrow 6}$ (6.5%)
Claude-O 4.1		129	7 (5.4%)	121 $^{\downarrow 8}$	9 $^{\uparrow 2}$ (7.4%)
Qwen-Coder		63	5 (7.9%)	76 $^{\uparrow 13}$	8 $^{\uparrow 3}$ (10.5%)
CodeLlama		19	13 (68.4%)	26 $^{\uparrow 7}$	4 $^{\downarrow 9}$ (15.4%)
DS-Coder		41	8 (19.5%)	42 $^{\uparrow 1}$	8 (19.1%)
Total	1,595	343	45 (13.1%)	357 $^{\uparrow 14}$	35 $^{\downarrow 10}$ (9.8%)

Correct Patches: the *higher* the better, Untrustworthy: the *lower* the better
Green and red arrows: improvement and deterioration, respectively

TABLE IV: **RQ1–VFXplainBench**. Comparing the untrustworthiness across the top 25 most dangerous CWE-IDs [42]

CWE-ID	Correct Patches	Untrustworthy	CWE-ID	Correct Patches	Untrustworthy
CWE-125	273	51 (18.7%)	CWE-122	19	0 (0.0%)
CWE-476	255	19 (7.5%)	CWE-20	14	1 (7.2%)
CWE-787	206	51 (24.8%)	CWE-200	13	0 (0.0%)
CWE-416	146	62 (42.5%)	Others	8	3 (37.5%)

the annotation may still be subjective sometimes and may contain labeling errors. We discuss this threat in Section VII.

E. RQ1: Benchmark Analysis

1) Untrustworthy Predictions: Table II shows that 17.2% of correct patch predictions, nearly one in five, are actually untrustworthy. Untrustworthiness occurs in every configuration, including the most advanced LLMs, GPT-4 and Claude-Opus 4.1. Hence, it is not an artifact of a particular model or dataset. This emphasises that correctness-based evaluation may inflate LLMs' repair capability. The untrustworthiness rate also varies across datasets: 11.4% on ExtractFix and ZeroDay, 18.4% on PrimeVul, and 24.9% on SVEN. This suggests that trustworthiness may depend on characteristics of vulnerabilities.

2) Vulnerability Types: We further compare the trustworthiness across different CWE-IDs. Table IV shows no untrustworthy predictions for CWE-122 (Heap-based Buffer Overflow) and CWE-200 (Exposure of Sensitive Information). Predictions for CWE-476 (NULL Pointer Dereference), CWE-20 (Improper Input Validation), CWE-125 (Out-of-bounds Read), and CWE-787 (Out-of-bounds Write) are considerably untrustworthy (7.5%–24.8%). The studied tools especially make a large portion of untrustworthy predictions for CWE-416 (Use After Free) and others (37.5%–42.5%). This finding further emphasises that trustworthiness depends on characteristics of vulnerabilities. In Appendix B-3, we discuss the untrustworthiness across single- and multi-hunk vulnerabilities. We find that untrustworthy predictions are more common in single-hunk vulnerabilities than in multi-hunk ones.

> **Takeaway #1:** Even when LLM-based AVR tools produce correct patches, 17.2% of their predictions are untrustworthy. The untrustworthiness varies across datasets (11.4%–24.9%) and vulnerability types (0%–42.5%).

3) Zero-shot vs. Few-shot: IP (zero-shot CoT prompting) and APPATCH (few-shot CoT prompting) use the same LLM backends and datasets. This allows us to compare their trustworthiness, as shown in Table III. APPATCH generates 14 more correct patches and 10 fewer untrustworthy predictions than IP, a 25.0% improvement in overall untrustworthiness rate. APPATCH generates more correct patches than IP with every LLM backend except Claude-O 4.1, and makes more trustworthy predictions with GPT-4 and CodeLlama. Surprisingly, APPATCH with Claude-O 4.1 and Qwen-Coder is less trustworthy, while DS-Coder remains unchanged.

We further compare the two prompting strategies using the Scott-Knott Effect Size Difference (ESD) test [43], which leverages hierarchical clustering to partition their means into statistically distinct groups. Results reveal APPATCH significantly decreases untrustworthy predictions, while the difference in correct patches is not significant. Detailed results can be found in Appendix B-2.

> **Takeaway #2:** APPATCH, by incorporating relevant contexts and few-shot CoT prompting, is statistically significantly more trustworthy than zero-shot CoT prompting.

4) Evaluation over LLMs: Table III shows that closed-source LLMs produce the most correct patches, especially Claude-O 4.1 with 129 by IP and 121 by APPATCH. Open-source LLMs generate fewer correct patches, with CodeLlama showing the worst repair success rate. Their trustworthiness also exhibits a similar trend. GPT-4 and Claude-O 4.1 have the lowest untrustworthiness rates (6.5% and 7.4%), compared to Qwen-Coder, CodeLlama, and DS-Coder (10.5%, 15.4%, and 19.1%). The Scott-Knott ESD test further confirms that these differences are statistically significant.

> **Takeaway #3:** Untrustworthiness occurs in all LLM backends, including the most advanced ones, such as GPT-4 and Claude-O 4.1. Closed-source LLMs are more effective and trustworthy in AVR than the open-source models.

5) Stochastic Nature of LLMs: As LLMs are stochastic, repeated predictions for the same vulnerability may differ in trustworthiness. To investigate this, we analyse Vul-R2's 10 runs per vulnerability. For each vulnerability that Vul-R2 fixes correctly in at least one run, we compute its trustworthiness rate[7]. A rate of 1.0 means every run is trustworthy, while 0.0 means Vul-R2 produced correct patches, but never with a correct explanation. Figure 2 shows the distribution of this rate on PrimeVul and SVEN. Trustworthiness is highly unstable across runs. The median vulnerability is fixed trustworthily in only 3/10 runs on PrimeVul and 2/10 on SVEN. Detailed analysis can be found in Appendix B-4.

[7]The fraction of 10 runs that produce a trustworthy prediction

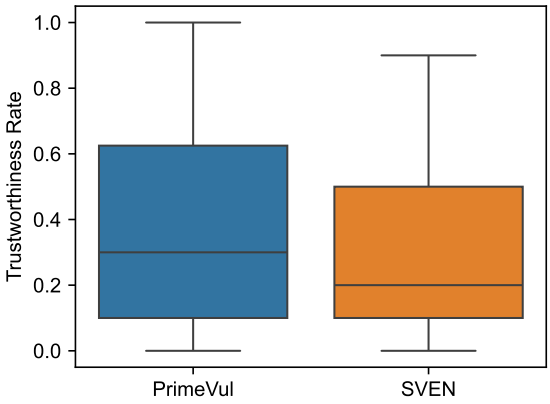


Fig. 2: The trustworthiness rate across 10 predictions made by Vul-R2 for the same vulnerability.

Takeaway #4: Trustworthiness is stochastic, as it may vary across runs. The median vulnerability is fixed trustworthily in only 3/10 runs on PrimeVul and 2/10 on SVEN.

6) Qualitative Analysis: We analysed 1,678 correct patch predictions in VFXplainBench. We first performed initial open coding [44] on a random set of 75 predictions for preliminary manual analysis. As mentioned in Section III-D, the annotators all documented their rationale and identified issues by referring to the NVD records. A prediction may be untrustworthy due to multiple issues. We summarised these rationales and categorised them into three types: Inconsistency between Explanation and Patch, Logical Inconsistency within Explanation, and Inconsistency between Explanation and Vulnerable Code.

Then, a different group of annotators independently annotated the remaining predictions following the same process, assessing whether the identified issues fit within the three types. Previously unknown issues were discussed to decide whether they could be accommodated by the existing types or required a new one. Ultimately, we identified three main types based on inconsistencies, along with a few rare cases.

1) **Inconsistency between Explanation and Patch** (52%). One of the most evident indicators of untrustworthy predictions is a mismatch between the patch and its explanation.
2) **Logical Inconsistency within Explanation** (67%). Annotators often assess the connection between two reasoning steps within the explanation: vulnerability analysis and the patching strategy. Some explanations show no logical connection between them. In some cases, the LLM even admitted and explicitly verbalised its errors or limitations.
3) **Inconsistency between Explanation and Vulnerable Code** (16%). The LLM hallucinates identifiers, e.g., variables or APIs [9], using "unknown" identifiers in the patch without any prior introduction in the explanation.

These types are not arbitrary patterns. Each represents a defect in the reasoning stated in the explanation w.r.t other AVR artifacts. Detecting such inconsistencies therefore detects defects in the explanation, motivating SUSVF introduced in Section IV. These inconsistency types are categorised based on the rationales documented by the annotators. At the same time, they do not fully define the labels. We also observed a few cases of untrustworthiness caused by unnatural code changes or awkward explanations. They do not fall into any of the given types (see Appendix C), which SUSVF fails and we leave them for future work to explore.

Takeaway #5: Inconsistencies can serve as a sufficient condition of untrustworthiness: Inconsistency between Explanation and Patch (52%), Inconsistency within Explanation (67%), and Inconsistency between Explanation and Vulnerable Code (16%).

IV. SUSVF: SPOTTING UNTRUSTWORTHY LLM-BASED VULNERABILITY FIXES

We formalise an LLM-based AVR tool as $M : x \mapsto (E, p)$, where x is the vulnerable code, and p is the generated patch. $E = (V, A, \cdot)$ is the explanation that contains an analysis V of the vulnerability and the fixing action A, both in NL. Applying the patch to the vulnerable code obtains the fixed code, $p(x)$. Based on findings in Section III-E6, we consider a prediction $x \mapsto (E, p)$ untrustworthy if it satisfies Equation 1.

$$(A \not\equiv p) \vee (A \not\Rightarrow \neg V) \vee \Big(i_{\text{use}}(p) \setminus i_{\text{def}}(p) \setminus i(E) \not\subseteq i(x)\Big) \quad (1)$$

Equation 1 comprises three components that capture inconsistencies across explanation–patch, logic within the explanation, and explanation–vulnerable code, respectively. Each provides concrete evidence of untrustworthiness. The first component, $A \not\equiv p$, represents cases where the explanation, particularly the fixing action A, is inconsistent with the generated patch p. The second component, $A \not\Rightarrow \neg V$, captures hallucinated reasoning steps with conflicting information: the fixing action A cannot eliminate the vulnerability analysed in V. The last component, $i_{\text{use}}(p) \setminus i_{\text{def}}(p) \setminus i(E) \not\subseteq i(x)$, describes the inconsistency between the explanation and the vulnerable code in identifiers $i(\cdot)$. The patch p may use identifiers $i_{\text{use}}(p)$ that are neither defined $i_{\text{def}}(p)$, introduced in the explanation $i(E)$, nor exist in the vulnerable code $i(x)$.

Equation 1 does not depend on how each relation is assessed. Implementing this assessment is nonetheless challenging, because each relation compares artifacts of different types, and a naive check would be unreliable. To this end, we propose SUSVF, an automated approach that detects untrustworthy predictions based on inconsistencies. Figure 3 illustrates SUSVF's three-stage process. SUSVF takes as input the vulnerable code, the explanation, and the generated patch. Each stage verifies the corresponding inconsistency. *A prediction is considered untrustworthy if it satisfies any constraints representing the inconsistencies* in Equation 1.

A. Assessing Inconsistency between Explanation and Patch

A prediction is untrustworthy if the explanation, specifically the fixing action A, fails to justify the generated patch p, i.e., A incorrectly describes p. We identify such cases by measuring their semantic equivalence. However, this is non-trivial, as p is written in a programming language, whereas A is in NL, and To address this, SUSVF transforms p into an NL description, $p \mapsto d_p \mid p \equiv d_p$. estimates its semantic equivalence with A.

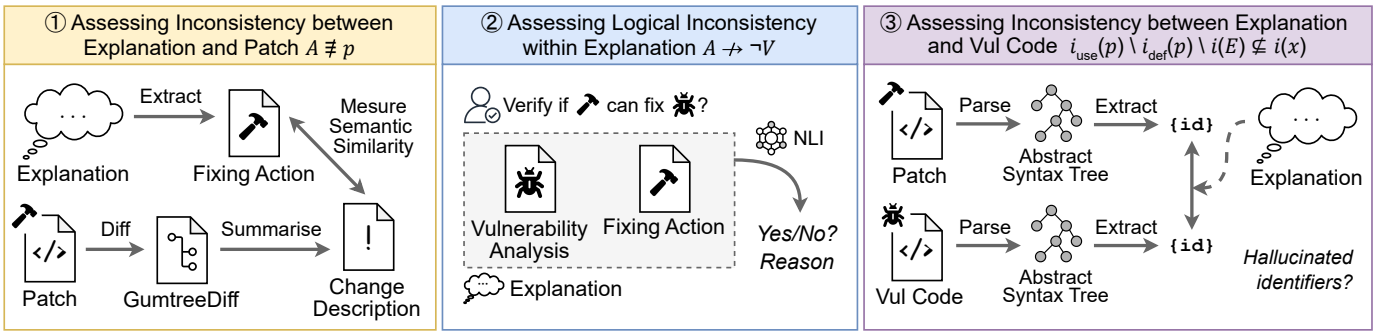


Fig. 3: SUSVF's process consists of three stages. Stage ① assesses the inconsistency between the explanation and the patch, based on the semantic equivalence between the explanation and the change description. Stage ② assesses the explanation's logical inconsistency, particularly between the vulnerability analysis and fixing action. Stage ③ assesses the inconsistency between the explanation and the vulnerable code.

Change Description Generation. This step generates an NL change description to derive the same type of representation for comparison with the fixing action. We transform this task into a summarization task, i.e., summarizing code changes in the patch into an NL description. The raw patch records coarse-grained, line-level textual diffs. Although it can be used directly, doing so has several limitations. In particular, some changes occur at the token level, such as identifier renaming or operator replacement. As a result, the raw patch can be less explicit in highlighting these changes.

To address this, we extract fine-grained differences between the vulnerable and patched code using specialised differencing tools. Specifically, we leverage Gumtree [17], which parses both code snippets and finds fine-grained changes between them based on their abstract syntax trees (ASTs). This allows us to explicitly capture small changes more directly at the AST node level. Gumtree breaks code changes down into four types of operations: add, remove, update and move on AST nodes. We then leverage a natural language inference (NLI) machine, such as an LLM, due to its capabilities, to summarise Gumtree's output into an NL description of code changes, denoted by d_p. Gumtree's AST-level output helps the NLI machine focus on the actual changes, compared to the raw patch describing changes at the line level.

Figure 4 illustrates this process for the patch shown in Figure 1. Gumtree finds that the only change is adding a new subcondition `*out_len < cipher_len-2` into the if condition. SUSVF feeds Gumtree's output into an NLI machine, e.g., GPT-4o-mini, and instructs it to summarise the changes into "*Adding a subcondition to ensure that *out_len is not less than cipher_len-2*." Compared to direct summarisation without using Gumtree, this description is more precise and pinpoints the exact change used to fix the vulnerability.

Semantic Similarity Comparison. To assess patch–explanation inconsistency, we compare the change description d_p and the fixing action A based on their semantic equivalence. We leverage several metrics following prior work [9].

1) Rouge [45] score measures the similarity between two summaries using overlapping n-grams. Following Ullah et al. [9], we use the threshold of 0.34 and mark A and d_p as similar if their Rouge score exceeds this threshold.

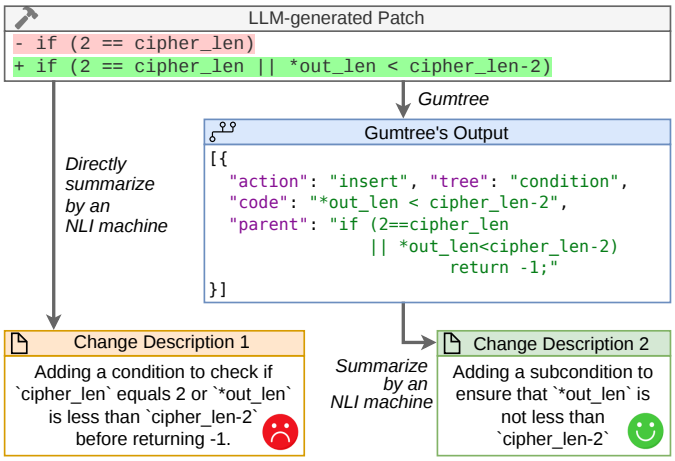


Fig. 4: Generating NL descriptions by using an LLM to summarise fine-grained code changes identified by Gumtree [17].

2) Cosine Similarity between the embeddings of A and d_p generated by OpenAI's "*text-embedding-3-small*". We use the optimal threshold of 0.84 identified by Ullah et al. and consider A and d_p similar if their cosine similarity exceeds this value.
3) An LLM is used as a judge to evaluate if A and d_p align. If scoring 1 if it responds "yes" and 0 otherwise.

SUSVF assesses whether A and d_p describe the same change, not whether they use the same wording. The three metrics capture similarity in changes at different levels. Rouge measures lexical overlap, cosine similarity measures closeness in an embedding space, and the LLM judge assesses whether the two descriptions align in meaning. As these metrics may be biased, we apply majority voting [46]. Each metric votes on whether $A \not\equiv p$, and the outcome supported by the most metrics wins. This design choice is evaluated in Section V-D.

B. Assessing Logical Inconsistency within Explanation

The explanation of a trustworthy prediction must be logically consistent. Intuitively, the fixing action A needs to eliminate the vulnerability trigger analysed in V. Once A is applied, the vulnerability trigger is no longer satisfied. If the

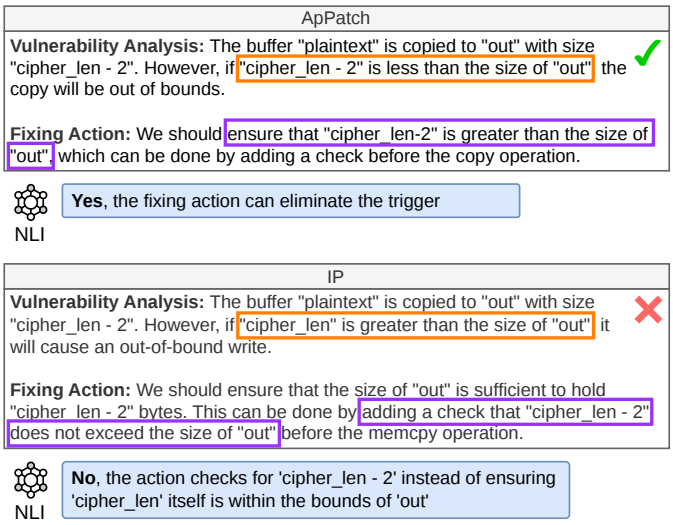


Fig. 5: Evaluating the logical inconsistency within the explanations provided by APPATCH and IP for fixing CVE-2018-16421. The fixing action (purple), the vulnerability trigger (orange), and their consistency verification are identified by an NLI machine, e.g., GPT-4o-mini.

explanation violates this constraint, $A \not\rightarrow \neg V$ as described in Section III-E6, it is logically inconsistent.

For example, the explanation in Figure 1 includes the vulnerability trigger "*if cipher_len is less than 2, plaintext will be out of bounds*" and the fixing action "*ensure that cipher_len is greater than 2, which can be done by adding a check before the copy operation*". As the action eliminates the trigger, this explanation seems logically consistent. Note that this prediction is still untrustworthy due to the inconsistency between the patch and the explanation.

As V and A are explicitly stated in NL, we frame the assessment of whether A can eliminate V as an NLI task. Unlike AVR, which requires substantial domain knowledge, this task inherently assesses the logical consistency, in particular logical entailment, between two NL statements. The question is not whether the two steps are phrased alike or described at the same level of detail. A fixing action described in general terms therefore passes as long as it addresses the stated trigger. In contrast, an incomplete explanation that does not explain how the program is vulnerable and how to fix it is inconsistent.

We leverage LLMs' NLI capabilities to address this task. An LLM is instructed to follow four outlined steps with few-shot learning. Based on the manual annotation described in Section III, we select four representative explanations, two logically consistent and two logically inconsistent, which are sufficient for effective few-shot learning [47]. The prompt template can be found in Appendix 1.

Figure 5 illustrates this stage on two explanations provided by APPATCH and IP for fixing CVE-2018-16421, with A and V are highlighted in purple and orange, respectively. By instructing the LLM to evaluate whether A can eliminate V, it is revealed that the DS-APPATCH's explanation is consistent, while IP's is not, as its action only checks 'cipher_len–2' instead of ensuring 'cipher_len' is within the bounds of 'out'.

C. Assessing Inconsistency between Explanation and Code

The LLM may hallucinate and use "unknown" variables or APIs in the patch that are not defined or introduced in the explanation, raising trustworthiness concerns, as described in Section III-E6. The vulnerable code and the explanation are deemed inconsistent, $i_{\text{use}}(p) \setminus i_{\text{def}}(p) \setminus i(E) \not\subseteq i(x)$, if the patch uses such hallucinated identifiers. We employ a lightweight approach to detect them.

SUSVF first parses the vulnerable code and the patched code into ASTs using the parser available in Gumtree. We then extract all identifiers in the vulnerable code and the patched code by traversing the ASTs and applying rule-matching based on the type of the current AST node and its parent. SUSVF then finds "unknown" identifiers that are absent from the vulnerable code by comparing two sets of identifiers. For each "unknown" identifier, we traverse the patched code's AST to verify if there exists a corresponding declaration node. A final search is conducted in the explanation to filter out identifiers introduced or defined in the explanation without literal code. A non-empty list indicates that the LLM is likely to hallucinate when making the AVR prediction.

V. EVALUATION OF SUSVF

This section describes a series of experiments conducted to evaluate how well SUSVF detects untrustworthy predictions made by LLM-based AVR systems.

A. Experimental Setup

In addition to RQ1 in Section III, we also study the following RQs:

RQ2 [Effectiveness]. *How effective is* SUSVF *at identifying untrustworthy predictions among correct patch predictions?* We frame this task on VFXplainBench as a binary classification problem with trustworthy and untrustworthy as negative and positive labels, respectively. We then compare the effectiveness of SUSVF against LLM-as-a-Judge baselines.

RQ3 [Efficiency]. *How efficient is* SUSVF *in identifying untrustworthy predictions?* We focus on the cost-efficiency of SUSVF by analyzing both processing time (in seconds) and the monetary cost (in USD) of using LLM APIs per prediction.

RQ4 [Ablation]. *How do different design choices impact* SUSVF*?* We compare SUSVF with variants that summarise code changes directly from raw patches without Gumtree, that use a single metric instead of majority voting to assess patch–explanation inconsistency, and that omit each stage of SUSVF.

RQ5 [Sensitivity]. *How do different NLI machines affect* SUSVF*?* We observe how SUSVF changes when using different LLMs as the NLI machine. For LLM-as-a-Judge baselines, we employ strong commercial models available at the time of writing, namely GPT-4 and Claude-Opus 4.1, to achieve the best performance. In contrast, for SusVF's NLI machine, due to resource constraints, we intentionally use lightweight and cost-efficient models from the same families, including GPT-3.5-turbo, GPT-4o-mini, and Claude-Haiku-4.1. For open-source LLMs for NLI, we select the strongest-performing family in VFXplainBench, Qwen2.5-Coder, as shown in Table III, and evaluate multiple model scales (1.5B, 7B, 14B, and 32B) to examine the impact of model size.

Metrics: For effectiveness, we use standard binary classification metrics: *Precision*, *Recall* (TPR), *Specificity* (TNR), and *Matthews Correlation Coefficient* (MCC). Precision measures the proportion of truly untrustworthy predictions among detected ones. Recall measures the proportion of untrustworthy predictions that are detected. Specificity measures the proportion of trustworthy predictions correctly recognised. MCC is a balanced metric for binary classification: 1 indicates perfect classification, 0 corresponds to random classification, and –1 reflects inverse classification. For efficiency, we report the processing time (in seconds) and API cost (in USD).

Baseline: We compare SUSVF against LLM-as-a-Judge [25], using all five LLMs shown in Table I, each denoted by the prefix Judge–. These include GPT-4 and Claude-Opus 4.1, the two strongest models available to us at the time of writing, as well as three open-source LLMs. We access GPT-4 and Claude-O 4.1 remotely using OpenAI and Anthropic APIs, respectively, while the open-source LLMs are deployed locally via HuggingFace APIs.

For a fair comparison, each baseline receives the vulnerable code, the patch, and the explanation, exactly as SUSVF does. Each baseline's prompt combines a security-auditor role, CoT reasoning, and a step-by-step decomposition mirroring SUSVF's three stages. Although the baselines are single-call, CoT prompting makes them reason through each step before answering. The prompt template is presented in Appendix 2. Claude-O 4.1 is the only LLM that has a thinking mode, and we use the default standard setting.

Dataset: We use 1,678 correct patch predictions in VFXplainBench to evaluate SUSVF. These predictions cover different types, including single-function, multi-function, single-hunk, and multi-hunk [19], [21]. SUSVF is not inherently limited to single-function, single-hunk vulnerabilities. This is because Gumtree, the static analysis tool we use, can parse a whole repository and find code differences at the repository level, while NLI is code-agnostic.

Independence between Labels and Detection. One may ask whether the evaluation is circular, i.e., the manual labeling criteria are simply SUSVF's detection rules. We argue that it is not for three reasons. First, the labels were assigned before SUSVF was designed, by judging each explanation against ground-truth references, as shown in Section III-D2. The inconsistency types were derived from the labeled predictions afterwards, not the other way around. Second, SUSVF never sees the ground-truth references. Its agreement with the labels thus shows that the reference-free inconsistency-based detection can recover judgments made by annotators with access to ground truth. Third, the labels are not defined by the three inconsistency types. Some untrustworthy predictions exhibit none of them, as discussed in Section III-E6.

Implementation: The explanations of the studied AVR systems are mostly formatted and structured. They explicitly describe the vulnerability analysis and the fixing action. Based on the format of each system, we use regex to extract the two components. For unstructured explanations, we use an NLI machine to extract them, with the prompt template described in Appendix 1. We use GPT-4o-mini as SUSVF's default NLI machine, then analyse the impact of different LLMs in RQ5.

TABLE V: **RQ2**. The effectiveness of SUSVF and the LLM-as-a-Judge baselines on VFXplainBench

Method	Pre	Rec	Spe	MCC
SusVF (ours)	0.882	0.732	0.980	0.768
Judge-GPT-4	0.251‡	0.404‡	0.751‡	0.131‡
Judge-Claude-O-4.1	0.356‡	0.544‡	0.797‡	0.292‡
Judge-Qwen-Coder	0.401‡	0.261‡	0.919‡	0.216‡
Judge-CodeLlama	0.238‡	0.139‡	0.908‡	0.059‡
Judge-DS-Coder	0.128‡	0.017‡	0.976‡	-0.018‡

and : the best and second best values in each column, respectively
Two-sided t-test †: p-value < 0.05, ‡: p-value < 0.01

We set all LLMs' temperature to 0. Experiments are conducted on a Rocky Linux server, 24 cores of Intel(R) Xeon(R) Gold 6150 CPU @ 2.70GHz, 256GB RAM, NVIDIA A100 GPU, 40GB memory.

B. RQ2: Effectiveness

Compared with Baselines. Table V shows that the LLM-as-a-Judge baselines perform poorly. While they show high Specificity of 0.751–0.976, their Precision (0.128–0.401) and Recall (0.017–0.544) are relatively low. This means that they often fail to understand the nuances of untrustworthiness and misclassify predictions as trustworthy. Their low MCC (-0.018–0.292) further indicate a negligible to weak correlation between their judgements and actual untrustworthiness. This suggests that LLM-as-a-Judge approaches provide unreliable assessments of prediction trustworthiness. In addition, we find no meaningful differences between self-validation, i.e., the same LLM evaluating its own predictions, and cross-validation, that is, evaluating another LLM's predictions.

In contrast, SUSVF consistently demonstrates strong effectiveness, with Precision of 0.882, Recall of 0.732, and Specificity of 0.980. Its MCC of 0.768 also shows a strong correlation with untrustworthiness. Compared to the baselines, SUSVF improves by up to 589% in Precision, 4,206% in Recall, 31% in Specificity, and 4,367% in MCC. All improvements are statistically significant.

Robustness to Superficial Differences. A potential concern is that differences in how explanations are expressed, such as wording, length, or granularity, may be mistaken for inconsistencies. Our results indicate that this rarely happens. The 1,389 trustworthy predictions in VFXplainBench come from 11 system configurations, and their explanations vary widely in these respects. SUSVF's Specificity is 0.980, i.e., only about 2% of trustworthy predictions are misclassified.

Top 25 Most Dangerous CWE-IDs. We report SUSVF's effectiveness across the top 25 most dangerous CWE-IDs [42] in Appendix B-1. SUSVF maintains Precision of 0.808–1, Recall of 0.733–1, and Specificity of 0.887–1. However, its effectiveness is more moderate for certain vulnerabilities (e.g., CWE-125). This indicates that some types are more challenging for detecting untrustworthy predictions.

Single-Hunk vs. Multi-Hunk. Appendix B-3 reports SUSVF's effectiveness across single- and multi-hunk vulnerabilities. Overall, SUSVF performs consistently well in both cases, with the effectiveness on single-hunk vulnerabilities slightly higher by 7% in MCC.

TABLE VI: **RQ3–RQ4**. The efficiency of SUSVF on VFXplainBench. We report the average processing time (in seconds) and API's cost (in $) per prediction

(a) Compare w/ baselines

Method	Time	API
SusVF (ours)	1.78	0.001
Judge-GPT-4	4.73	0.044
Judge-Claude-O-4.1	12.57	0.059
Judge-Qwen-Coder	31.62	-
Judge-CodeLlama	24.80	-
Judge-DS-Coder	19.05	-

(b) Ablation study

Method	Time	API
w/o Gumtree	1.71	0.001
w/ only Rouge	1.73	0.001
w/ only Cosine Sim.	1.70	0.001
w/ only An LLM	1.63	0.001
w/o ① Expl–Patch	1.29	0.001
w/o ② Expl's Logic	1.46	0.001
w/o ③ Expl–Code	1.75	0.001

Effectiveness Across AVR Systems. The effectiveness of SUSVF on different LLM-based AVR systems is also presented in Appendix B-5. We evaluated 11 system settings: Vul-R2 and 10 combinations of five LLM backends with two approaches (IP and ApPatch). For most settings, SUSVF achieves Precision of 0.750–1, Recall of 0.625–0.846, Specificity of 0.966–1, and MCC of 0.713–0.852. For IP w/ Qwen-Coder and APPATCH w/ CodeLlama (only 4–5 untrustworthy examples), results are relatively lower, with Precision of 0.6, Recall of 0.5–0.6, and MCC of 0.549–0.677.

Case Study. We report and analyze a few cases of false positives (FPs) and false negatives (FNs) in Appendix C. Overall, FPs mainly occur because Stage ① cannot capture the semantic equivalence between the patch and the explanation in challenging cases. Another reason is that NLI in Stage ② may fail when vulnerability conditions and fixing actions are too complex. FNs are mostly due to NLI hallucinations. Our current implementation uses a lightweight, cost-efficient LLM (i.e., GPT-4o-mini) as the NLI machine. We believe stronger LLMs can reduce FPs and FNs.

Takeaway #6: LLM-as-a-Judge baselines fail to detect untrustworthy AVR predictions. In contrast, SUSVF consistently and significantly outperforms the baselines across all metrics. It demonstrates a strong correlation with untrustworthiness and maintains good effectiveness across AVR predictions and systems.

C. RQ3: Efficiency

Table VIa reports SUSVF's efficiency in handling a prediction, measured by average processing time (in seconds) and the cost of using the APIs (in $). On average, per prediction, SUSVF requires 1.78 seconds and $0.001 due to the use of OpenAI's APIs for the NLI machine. Compared to the LLM-as-a-Judge methods, SUSVF achieves the best balance of time and $. All baselines are slower than SUSVF, requiring 4.73–31.62 seconds per prediction. Commercial LLMs, Judge-GPT-4 and Judge-Claude-O-4.1, are also 44–59 times more expensive ($0.044–$0.059). In contrast, open-source LLMs, Qwen-Coder, CodeLlama, and DS-Coder, deployed locally as judges, incur no API cost but are much slower (19.05–31.62 seconds). While the monetary cost of invoking these LLMs' APIs is 0, there are indirect costs, e.g., electricity and GPU, which we do not count in this study. As the remote LLMs run on more powerful machines than our local deployment for the open-source LLMs, as described in Section V-A, it is expected that these local LLMs are less time-efficient.

Takeaway #7: SUSVF requires 1.78 seconds and $0.001 for using GPT-4o-mini as the NLI machine per prediction. It is substantially more money-efficient than commercial LLMs-as-a-Judge baselines, while also being far faster than locally deployed ones.

TABLE VII: **RQ4**. Comparing SUSVF with its ablated versions: an approach directly using raw patches without relying on Gumtree, three approaches that use a single metric to assess patch–explanation inconsistency, and three approaches that exclude each stage in the three-stage process of SUSVF

Method	Pre	Rec	Spe	MCC
SusVF (ours)	0.882	0.732	0.980	0.768
w/o Gumtree	$0.598^{\dagger}$	$0.659^{\ddagger}$	$0.909^{\dagger}$	$0.546^{\ddagger}$
w/ only Rouge	$0.539^{\ddagger}$	$0.822^{\dagger}$	$0.855^{\ddagger}$	$0.580^{\ddagger}$
w/ only Cosine Similarity	$0.262^{\ddagger}$	$0.948^{\dagger}$	$0.449^{\ddagger}$	$0.309^{\ddagger}$
w/ only An LLM	$0.552^{\ddagger}$	$0.815^{\dagger}$	$0.863^{\ddagger}$	$0.588^{\dagger}$
w/o ① Expl–Patch	$1.000^{\ddagger}$	$0.551^{\ddagger}$	$1.000^{\ddagger}$	0.709
w/o ② Expl's Logic	$0.750^{\ddagger}$	$0.293^{\ddagger}$	$0.960^{\ddagger}$	$0.411^{\ddagger}$
w/o ③ Expl–Code	$0.879^{\ddagger}$	0.707	$0.960^{\ddagger}$	$0.750^{\ddagger}$

: the best value in each column, : the worst value in each column
Two-sided t-test †: p-value < 0.05, ‡: p-value < 0.01

D. RQ4: Ablation Study

Effectiveness: We first compare the effectiveness of SUSVF with a variant that summarises LLM-generated patches without using Gumtree (w/o Gumtree). As shown in Table VII, excluding Gumtree overall reduces the effectiveness, with statistically significant drops by 32% in Precision, 10% in Recall, 7% in Specificity, and 29% in MCC. The results show that leveraging Gumtree can significantly improve SUSVF's detection of untrustworthiness.

We also examine the impact of using a single metric to assess semantic equivalence between the patch and its explanation in Stage ①, rather than an ensemble of metrics. As shown in Table VII, although all three variants of SUSVF achieve higher Recall (by 11%–30%), their Precision, Specificity, and MCC decrease significantly. In particular, the variant using only cosine similarity performs the worst, with reductions of 70% in Precision, 54% in Specificity, and 60% in MCC. These results indicate that a single metric is insufficient for reliably assessing semantic equivalence, leading to a bias toward incorrectly classifying predictions as untrustworthy.

Table VII also compares SUSVF's effectiveness with its ablated versions, each omitting one stage of the three-stage process: assessing the inconsistency between the explanation and the patch, assessing the explanation's logical inconsistency, or assessing the inconsistency between the explanation and the vulnerable code. Removing Stage ① statistically significantly drops Recall by 25%. MCC also drops by 8%; however, this difference is not statistically significant. Both Precision and Specificity reach 1, indicating that SUSVF is now biased toward labeling predictions as trustworthy. Removing Stage ② leads to statistically significant decreases in

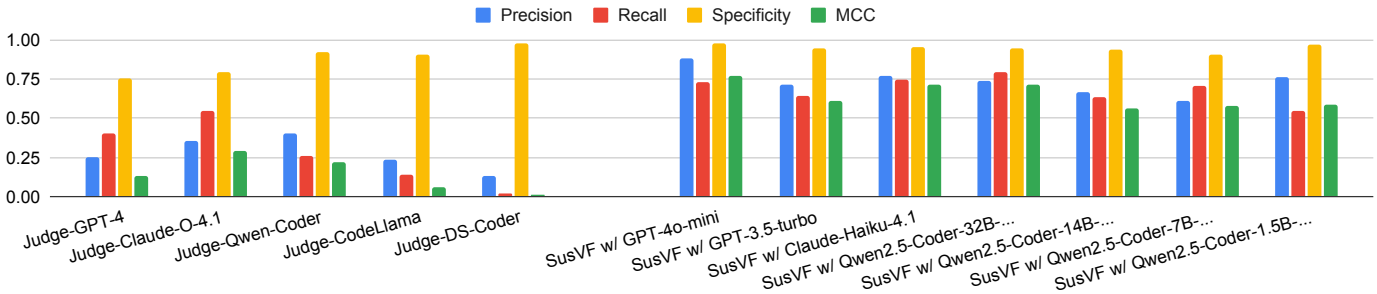


Fig. 6: **RQ5**. The effectiveness of LLM-as-a-Judge and SusVF (ours) with different LLMs as the NLI machine.

all metrics. Particularly, it drops Precision by 15%, Recall by 60%, Specificity by 2%, and MCC by 47%. Removing Stage ③ also decreases SusVF's effectiveness by 1% in Precision, 3% in Recall, 2% in Specificity, and 2% in MCC. These differences are statistically significant, except for Recall.

> 👉 **Takeaway #8:** Gumtree, the ensemble of metrics in Stage ①, and all three stages play important roles in SusVF's effectiveness. Excluding them leads to statistically significant decreases by up to 60% in MCC.

Efficiency: Table VIb shows that using Gumtree, the ensemble in Stage ①, and the last two stages have no big impact on SusVF's efficiency. The process time and the cost of using OpenAI's APIs per AVR prediction remain nearly unchanged. These components appear to be computationally lightweight. In contrast, Stage ① adds 0.49 seconds (28%) to processing time, indicating that most overhead comes from comparing change descriptions with explanations.

> 👉 **Takeaway #9:** Most of SusVF's overhead comes from comparing change descriptions with explanations (e.g., 28% of the total processing time). The other components in SusVF have a negligible impact on efficiency.

E. RQ5: Sensitivity Study

We investigate how the choice of LLMs used as the NLI machine influences SusVF's effectiveness. The results are shown in Figure 6. Details are presented in Appendix B-6.

Compared with LLM-as-a-Judge baselines, SusVF statistically significantly outperforms them across all NLI backends. Specifically, it achieves 52%–589% higher Precision, 0%–4,553% higher Recall, and 93%–4,367% higher MCC. Improvements in Specificity are more modest (up to 30%). Notably, SusVF with Qwen2.5-Coder-32B-Instruct also outperforms its corresponding Judge baseline.

Among the NLI machines, GPT-4o-mini, Claude-Haiku-4.1, and Qwen2.5-Coder-32B-Instruct achieve the highest effectiveness, with Precision of 0.737–0.882, Recall of 0.732–0.791, Specificity of 0.942–0.980, and MCC of 0.711–0.768. The other LLMs show relatively lower effectiveness, with Precision of 0.611–0.765, Recall of 0.544–0.707, Specificity of 0.907–0.966, and MCC of 0.581–0.610. While Qwen2.5-Coder-32B-Instruct demonstrates the best effectiveness among the Qwen2.5-Coder-x-Instruct LLMs, we observe no big differences among the three smaller respective LLMs.

> 👉 **Takeaway #10:** SusVF with GPT-4o-mini, Claude-Haiku-4.1, and Qwen2.5-Coder-32B-Instruct achieve the best effectiveness. SusVF consistently outperforms LLM-as-a-Judge across all NLI machines, especially when the NLI machine and the LLM-as-a-Judge are the same model.

VI. Discussion

*1) **Implications for AVR:*** Trustworthiness in LLM-based AVR poses a fundamental challenge beyond patch correctness. Our findings demonstrate that patch correctness alone is insufficient for ensuring reliability. This is because a considerable proportion of correct patches can be generated with wrong explanations. This aligns with prior work, for example, Khati et al. [13] conducted a survey with developers and found that interpretability of generated outputs is clearly important for building trust. Together, these findings call for a paradigm shift in how LLM-based AVR systems are designed, trained, and evaluated, with a stronger emphasis on human–LLM alignment. This was further emphasised by Zhou et al. [2]. Current AVR benchmarks mainly prioritise functional correctness, assuming that correct patches reflect sound reasoning. Our results invalidate this assumption and suggest that future benchmarks should explicitly capture trustworthiness. to enable a more in-depth evaluation of LLMs and discourage surface-level correctness.

*2) **Benefits of Detecting Untrustworthy Predictions:*** During the *development*, automated detection of untrustworthy predictions plays a central role in assessing and improving system quality and reliability. It facilitates the review of AVR outputs, enabling automated trustworthiness testing to be integrated into continuous integration and delivery pipelines. This supports the iterative development process, where both performance and trustworthiness must be continuously evaluated and refined [33], [34]. In addition, untrustworthy predictions provide valuable diagnostic signals that help identify deeper behavioural issues in LLM reasoning and guide the improvements of LLM-based AVR systems.

During *deployment*, detection methods serve as a safeguard by flagging untrustworthy predictions in real time and reducing the risk of misleading or unsafe repairs. This is particularly important in security-sensitive environments, where incorrect vulnerability fixes may have severe consequences. Hence, these methods foster greater human trust in AVR systems.

*3) **Limitations and Future Work**:* SUSVF encodes three inconsistency criteria derived from our analysis of VFXplainBench. Nevertheless, a few cases do not fit into these criteria. This is because inconsistency is a *sufficient* but not necessary condition for untrustworthiness. If the explanation is inconsistent, the prediction is untrustworthy, but not vice versa. Appendix C presents a few FPs and FNs, and analyzes their potential causes. Future work can address these cases to further improve the effectiveness of SUSVF.

In addition, learning-based assessments in Stage ① and Stage ② could be inaccurate. For example, rouge and cosine similarity may be limited, while LLMs can hallucinate. We can strengthen SUSVF by incorporating advanced semantic measures and formal methods. For instance, vulnerability triggers and fixing actions could be encoded as logical formulas and analysed using Satisfiability Modulo Theories (SMT) solvers to assess their consistency. Exploring such techniques represents a promising direction for enhancing SUSVF.

VII. THREATS TO VALIDITY

Construct Validity. VFXplainBench builds on publicly available AVR resources and approaches that were carefully curated and validated in previous research [2], [18]–[21]. Our manual annotation may be subjective. To reduce this threat, we provided NVD-validated ground-truth references and instructed annotators to assess explanations against them. We also collected confidence scores and rationales, and used independent labeling by two annotators with a third resolving disagreements. As a result, we achieved a Cohen's Kappa of 0.84 for inter-reviewer agreement. These results suggest substantial consistency and reliability. We acknowledge that the labeling is imperfect, leaving a small portion of mislabeled data. However, we believe the results still hold good.

Another threat is circularity, since SUSVF is derived from the annotation used to evaluate it. We mitigate this threat in three ways: labeling was completed before SUSVF was designed, labels reflect correctness against ground-truth references rather than consistency, and SUSVF is reference-free.

Internal Validity. A threat can be the bias of the three metrics, rouge, cosine similarity, and LLM, used to assess patch–explanation consistency. To minimize this threat, we apply majority voting to limit false positives. A further threat is that the tools and LLMs used in SUSVF, e.g., Gumtree and the NLI machine, may be inaccurate. We addressed this by following best practices [47] and validated tools in prior studies [17]. SUSVF also does not use LLM/NLI-based methods to assess trustworthiness as a whole, but only for narrowly scoped subtasks, such as summarising Gumtree code differences into an NL change description. These subtasks are designed to be less complex than the complete trustworthiness decision and can be effectively handled by SOTA LLMs [48], [49]. However, we acknowledge that some bias may remain in some cases. In this respect, future empirical investigations may provide further insights.

A related threat is that SUSVF may mistake superficial differences, such as paraphrasing, a different level of abstraction or granularity, for genuine inconsistencies. We mitigate this threat by focusing on semantic entailment and using majority voting before raising inconsistencies. Specificity of 0.980 on VFXplainBench, whose explanations vary widely in style, indicates that such misclassifications are rare. Nevertheless, the measurement is not perfect. Appendix C shows false positives can still arise. We can mitigate this using stronger semantic comparison methods, as discussed in Section VI-3.

External Validity. One key threat is VFXplainBench's diversity. Although it was constructed from existing vulnerability datasets and AVR systems validated in previous research [19], [21], it may not fully capture the diversity of vulnerabilities. While this remains a limitation, please note that VFXplainBench includes predictions made by multiple AVR systems using various LLM backends over 57 vulnerability types across various projects. This provides a degree of diversity that supports the generalizability of our findings.

Since most vulnerability repair datasets are in C/C++, our experiments also focused on C/C++. Despite this, we believe SUSVF would still work with other languages. The static analysis tool, Gumtree, supports multiple languages, such as Java, JavaScript, Python, R, and Ruby. The LLM/NLI-based methods are language-agnostic.

In non-security program repair settings, we believe SUSVF still operates the same in theory. In such settings, the LLM still produces a patch and an explanation. SUSVF does not rely on the specification of vulnerability types. Therefore, it can be easily applied to non-security program repair tasks.

Still, broader studies covering more models, languages, settings, and vulnerabilities may reveal further insights into the trustworthiness of LLM-based AVR.

Conclusion Validity. We interpreted the results using multiple well-established metrics: Precision, Recall, Specificity, and MCC, and appropriate statistical tests, the Scott-Knott ESD test and the two-sided t-test, to support our findings.

VIII. RELATED WORK

LLM-based AVR. LLMs trained using billions of tokens can be directly leveraged for AVR without relying on vulnerability-fixing datasets. LLMs have substantially advanced AVR, outperforming existing AVR techniques. Prior studies [2] have identified two major categories of LLM-based AVR systems: (1) fine-tuning, which updates LLM parameters with labelled datasets, and (2) prompt engineering, which keeps parameters fixed while designing prompts to guide responses.

Regarding fine-tuning, studies have incorporated additional inputs beyond the vulnerable code to enhance AVR, such as ASTs, vulnerability descriptions, CWE examples, and related commits [2]. Other studies [35], [50] have also modified LLM architectures for better AVR. For example, Wen et al. [21] fine-tuned a reasoning LLM called Vul-R2 for AVR. They combined a reasoning learning module and a curriculum-based verifiable rewarded training module to generate both patches and explanations.

Regarding prompt engineering, researchers have explored *zero-shot prompting* to repair vulnerabilities without training. Prompts often include the vulnerable code [51], [52], the vulnerability locations [21], and/or vulnerability-related

semantic details [53]. Other studies have also applied *few-shot prompting* by providing a few repair examples [54] or *CoT prompting* to instruct LLMs to follow several intermediate reasoning steps [20], [37], [55]. Various approaches, such as APPATCH [19], have been introduced to automatically retrieve relevant contexts to assist prompting.

Despite these advances, current paradigms mainly focus on generating correct patches without examining LLMs' decision-making process. Hence, it remains unclear whether LLMs interpret and repair vulnerabilities in a trustworthy manner.

Patch Correctness and Overfitting. The reliability of AVR has long been studied through the lens of patch correctness. Most AVR tools leverage test cases, which often results in plausible patches. While they pass the available test cases, the plausible patches may actually be incorrect, a problem called *overfitting*. Such patches can even break functionality that the test cases do not cover [5]. Existing studies [56] have confirmed overfitting in the generated patches of well-studied tools, although many of their patches were indeed correct. In addition, AVR has been identified as a combination of undecidable problems, so patch correctness is undecidable in general. These limitations have motivated a large body of work on automated patch correctness assessment [7].

In the context of LLM-based AVR, new concerns have arisen. LLM-based tools can generate correct patches but rely on memorization [10] and spurious features unrelated to the underlying vulnerability [8]. This can inflate the measured repair capability of LLMs. For that reason, patch correctness assessment is inadequate for evaluating LLM-based AVR and for establishing trust in practice. As found by Noller et al. [12], developers expect evidence beyond correctness before they trust automatically generated patches.

Prior studies mainly assess whether a generated patch is correct. Our work, on the other hand, is complementary to these prior studies. It evaluates the repair capability of LLMs not only at the superficial level of correctness but also at the level of the decision-making process described in explanations.

Trustworthiness of LLMs in SE. LLMs have been rapidly applied in both industry and academia for various tasks. As they become increasingly integral to critical tasks, ensuring their trustworthiness is essential. Prior studies [57] have conceptualised trustworthiness across multiple dimensions, such as safety, fairness, explainability, ethics, robustness, and privacy. Following such concepts, researchers [14], [15] have raised concerns about LLMs' trustworthiness in various domains and proposed countermeasures.

In SE, studies [13], [29] have also defined trustworthiness and distinguished it from trust. Khati et al. [13] conducted a human study across multiple SE tasks, including program repair, and highlighted interpretability alongside accuracy as the two most critical factors. Unfortunately, existing work has shown that LLMs often struggle with interpretability in vulnerability analysis. Ullah et al. [9] showed that LLMs cannot reliably identify and reason about vulnerabilities, as they provide incorrect and unfaithful reasoning.

Recent efforts to evaluate and improve LLMs have mostly focused on accuracy and robustness. Li et al. [58] introduced SV-TrustEval-C, a benchmark that evaluates LLMs' abilities of vulnerability analysis under diverse data- and control-flow complexities and structural and semantic perturbations. Their results show that current LLMs are far from satisfactory, relying more on pattern matching than on robust reasoning.

Although situated in the domain of LLM-based AVR trustworthiness, our work is the first to systematically evaluate trustworthiness. Existing benchmarks and systems primarily assess *what* predictions LLMs produce (e.g., correctness, robustness) but rarely examine *why* those predictions are made. To bridge this gap, we propose VFXplainBench, a human-validated trustworthiness benchmark, and SUSVF, the first approach that detects untrustworthy AVR predictions.

IX. CONCLUSION

This paper introduced VFXplainBench, a benchmark of 11,210 predictions made by three AVR systems, namely instruction prompting, APPATCH, and Vul-R2. VFXplainBench revealed that a considerable portion is untrustworthy, even when the generated patches are correct. This motivated us to propose SUSVF, an automated approach that detects untrustworthy predictions among correct patch predictions by analysing inconsistencies between patch, explanation, vulnerable code, and reasoning steps. Results on VFXplainBench show that SUSVF significantly outperforms LLM-as-a-Judge baselines, demonstrating its effectiveness and efficiency. These findings underscore the importance of evaluating LLMs not only for their correctness but also for trustworthiness, and highlight the potential of automated explainability-based methods to accelerate auditing and foster safer deployment of LLM-based AVR. Future work can explore rare cases and incorporate formal methods to strengthen SUSVF.

REFERENCES

[1] X. Zhou, K. Kim, B. Xu, D. Han, and D. Lo, "Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3639222

[2] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, Dec. 2024, just Accepted. [Online]. Available: https://doi.org/10.1145/3708522

[3] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1482–1494.

[4] A. Nilizadeh and G. T. Leavens, " Be Realistic: Automated Program Repair is a Combination of Undecidable Problems ," in *2022 IEEE/ACM International Workshop on Automated Program Repair (APR)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2022, pp. 31–32. [Online]. Available: https://doi.ieeecomputersociety.org/10.1145/3524459.3527346

[5] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, "Is the cure worse than the disease? overfitting in automated program repair," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE 2015. New York, NY, USA: Association for Computing Machinery, 2015, p. 532–543. [Online]. Available: https://doi.org/10.1145/2786805.2786825

[6] X. B. D. Le, F. Thung, D. Lo, and C. L. Goues, "Overfitting in semantics-based automated program repair," *Empirical Software Engineering*, vol. 23, no. 5, pp. 3007–3033, Oct 2018. [Online]. Available: https://doi.org/10.1007/s10664-017-9577-2

[7] Z. Fei, J. Ge, C. Li, T. Wang, Y. Li, H. Zhang, L. Huang, and B. Luo, "Patch correctness assessment: A survey," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, Jan. 2025. [Online]. Available: https://doi.org/10.1145/3702972

[8] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: unfaithful explanations in chain-of-thought prompting," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.

[9] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 862–880.

[10] J. Kong, X. Xie, and S. Liu, "Demystifying memorization in llm-based program repair via a general hypothesis testing framework," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, Jun. 2025. [Online]. Available: https://doi.org/10.1145/3729390

[11] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 24 824–24 837. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf

[12] Y. Noller, R. Shariffdeen, X. Gao, and A. Roychoudhury, "Trust enhancement issues in program repair," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 2228–2240. [Online]. Available: https://doi.org/10.1145/3510003.3510040

[13] D. Khati, Y. Liu, D. N. Palacio, Y. Zhang, and D. Poshyvanyk, "Mapping the trust terrain: Llms in software engineering - insights and perspectives," *ACM Trans. Softw. Eng. Methodol.*, Oct. 2025, just Accepted. [Online]. Available: https://doi.org/10.1145/3771282

[14] L. Nguyen Tung, S. Cho, X. Du, N. Neelofar, V. Terragni, S. Ruberto, and A. Aleti, "Automated trustworthiness oracle generation for machine learning text classifiers," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, Jun. 2025. [Online]. Available: https://doi.org/10.1145/3729376

[15] L. N. Tung, X. Du, N. Neelofar, and A. Aleti, "Untrustvul: Automated untrustworthy alert identification in vulnerability detection models," *IEEE Transactions on Software Engineering*, vol. 52, no. 8, pp. 2335–2353, 2026.

[16] W. Han, Y. Kwak, M. Yu, K. Kim, Y. Lee, H. Moon, and Y. Paek, "Rethinking the capability of fine-tuned language models for automated vulnerability repair," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*, ser. ICSE '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1069–1082. [Online]. Available: https://doi.org/10.1145/3744916.3773167

[17] J.-R. Falleri and M. Martinez, "Fine-grained, accurate and scalable source differencing," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3639148

[18] Y. Li, F. H. Shezan, B. Wei, G. Wang, and Y. Tian, *SoK: towards effective automated vulnerability repair*. USA: USENIX Association, 2025.

[19] Y. Nong, H. Yang, L. Cheng, H. Hu, and H. Cai, *APPATCH: automated adaptive prompting large language models for real-world software vulnerability patching*. USA: USENIX Association, 2025.

[20] Y. Nong, M. Aldeen, L. Cheng, H. Hu, F. Chen, and H. Cai, "Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities," 2024. [Online]. Available: https://arxiv.org/abs/2402.17230

[21] X.-C. Wen, Z. Lin, Y. Yang, C. Gao, and D. Ye, "Vul-r2: A reasoning llm for automated vulnerability repair," in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '25, 2025. [Online]. Available: https://arxiv.org/abs/2510.05480

[22] X. Gao, B. Wang, G. J. Duck, R. Ji, Y. Xiong, and A. Roychoudhury, "Beyond tests: Program vulnerability repair via crash constraint extraction," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 2, Feb. 2021. [Online]. Available: https://doi.org/10.1145/3418461

[23] J. He and M. Vechev, "Large language models for code: Security hardening and adversarial testing," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1865–1879. [Online]. Available: https://doi.org/10.1145/3576915.3623175

[24] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, " Vulnerability Detection with Code Language Models: How Far are We? ," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 1729–1741. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/ICSE55347.2025.00038

[25] J. He, J. Shi, T. Y. Zhuo, C. Treude, J. Sun, Z. Xing, X. Du, and D. Lo, "From code to courtroom: Llms as the new software judges," 2025. [Online]. Available: https://arxiv.org/abs/2503.02246

[26] J. Long, "Large language model guided tree-of-thought," 2023. [Online]. Available: https://arxiv.org/abs/2305.08291

[27] Y. Yao, Z. Li, and H. Zhao, "Beyond chain-of-thought, effective graph-of-thought reasoning in language models," 2024. [Online]. Available: https://arxiv.org/abs/2305.16582

[28] A. Aleti, B. Ray, R. Hoda, and S. Chen, "Trustworthy ai software engineers," 2026. [Online]. Available: https://arxiv.org/abs/2602.06310

[29] L. Kastner, M. Langer, V. Lazar, A. Schomacker, T. Speith, and S. Sterz, " On the Relation of Trust and Explainability: Why to Engineer for Trustworthiness ," in *2021 IEEE 29th International Requirements Engineering Conference Workshops (REW)*. Los Alamitos, CA, USA: IEEE Computer Society, Sep. 2021, pp. 169–175. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/REW53955.2021.00031

[30] F. Liu, Y. Liu, L. Shi, Z. Yang, L. Zhang, X. Lian, Z. Li, and Y. Ma, " Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code ," *IEEE Transactions on Software Engineering*, vol. 52, no. 03, pp. 1037–1055, Mar. 2026. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/TSE.2026.3657432

[31] M. Böhme, E. O. Soremekun, S. Chattopadhyay, E. Ugherughe, and A. Zeller, "Where is the bug and how is it fixed? an experiment with practitioners," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE 2017. New York, NY, USA: Association for Computing Machinery, 2017, p. 117–128. [Online]. Available: https://doi.org/10.1145/3106237.3106255

[32] E. Winter, D. Bowes, S. Counsell, T. Hall, S. Haraldsson, V. Nowack, and J. Woodward, "How do developers really feel about bug fixing? directions for automatic program repair," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1823–1841, 2023.

[33] V. Riccio, G. Jahangirova, A. Stocco, N. Humbatova, M. Weiss, and P. Tonella, "Testing machine learning based systems: a systematic mapping," *Empirical Software Engineering*, vol. 25, no. 6, pp. 5193–5254, Nov 2020. [Online]. Available: https://doi.org/10.1007/s10664-020-09881-0

[34] S. Martínez-Fernández, J. Bogner, X. Franch, M. Oriol, J. Siebert, A. Trendowicz, A. M. Vollmer, and S. Wagner, "Software engineering for ai-based systems: A survey," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 2, Apr. 2022. [Online]. Available: https://doi.org/10.1145/3487043

[35] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, and D. Phung, "Vulrepair: a t5-based automated software vulnerability repair," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 935–947. [Online]. Available: https://doi.org/10.1145/3540250.3549098

[36] Z. Yu, Z. Guo, Y. Wu, J. Yu, M. Xu, D. Mu, Y. Chen, and X. Xing, *PATCHAGENT: a practical program repair agent mimicking human expertise*. USA: USENIX Association, 2025.

[37] Y. Kim, S. Shin, H. Kim, and J. Yoon, "Logs in, patches out: Automated vulnerability repair via {Tree-of-Thought}{LLM} analysis," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 4401–4419.

[38] D. Votipka, S. Rabin, K. Micinski, J. S. Foster, and M. L. Mazurek, "An observational investigation of reverse Engineers' processes," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 1875–1892. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/votipka-observational

[39] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=VTF8yNQM66

[40] J. Yang, Y. Wang, Y. Lou, M. Wen, and L. Zhang, "A large-scale empirical review of patch correctness checking approaches," in

Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ser. ESEC/FSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 1203–1215. [Online]. Available: https://doi.org/10.1145/3611643.3616331

[41] M. L. McHugh, "Interrater reliability: the kappa statistic," *Biochemia medica*, vol. 22, no. 3, pp. 276–282, 2012.

[42] MITRE Corporation, "2025 cwe top 25 most dangerous software weaknesses," https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html, 2025, accessed: 2026-01-26.

[43] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "The impact of automated parameter optimization on defect prediction models," *IEEE Transactions on Software Engineering*, vol. 45, no. 7, pp. 683–711, 2019.

[44] S. H. Khandkar, "Open coding," *University of Calgary*, vol. 23, no. 2009, p. 2009, 2009.

[45] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013/

[46] F. Leon, S.-A. Floria, and C. Bădică, "Evaluating the effect of voting methods on ensemble-based classification," in *2017 IEEE International Conference on INnovations in Intelligent SysTems and Applications (INISTA)*, 2017, pp. 1–6.

[47] U. Halil, J. Huang, D. Graux, and J. Z. Pan, "Llm shots: Best fired at system or user prompts?" in *Companion Proceedings of the ACM on Web Conference 2025*, ser. WWW '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1605–1613. [Online]. Available: https://doi.org/10.1145/3701716.3717814

[48] Q. Liu, S. Hyland, S. Bannur, K. Bouzid, D. Castro, M. Wetscherek, R. Tinn, H. Sharma, F. Pérez-García, A. Schwaighofer, P. Rajpurkar, S. Khanna, H. Poon, N. Usuyama, A. Thieme, A. Nori, M. Lungren, O. Oktay, and J. Alvarez-Valle, "Exploring the boundaries of GPT-4 in radiology," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 14 414–14 445. [Online]. Available: https://aclanthology.org/2023.emnlp-main.891/

[49] Y. Wang, M. Wang, and P. Nakov, "Rethinking STS and NLI in large language models," in *Findings of the Association for Computational Linguistics: EACL 2024*, Y. Graham and M. Purver, Eds. St. Julian's, Malta: Association for Computational Linguistics, Mar. 2024, pp. 965–982. [Online]. Available: https://aclanthology.org/2024.findings-eacl.65/

[50] M. Fu, V. Nguyen, C. Tantithamthavorn, D. Phung, and T. Le, "Vision transformer inspired automated vulnerability repair," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 3, Mar. 2024. [Online]. Available: https://doi.org/10.1145/3632746

[51] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, " Examining Zero-Shot Vulnerability Repair with Large Language Models ," in *2023 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2023, pp. 2339–2356. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP46215.2023.10179420

[52] B. Ahmad, S. Thakur, B. Tan, R. Karri, and H. Pearce, "On hardware security bug code fixes by prompting large language models," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 4043–4057, 2024.

[53] Y. Luo, B. Li, A. Singhal, P. Tseng, L. Zhang, Q. Zou, X. Sun, and P. Liu, "Exploring prompt patterns for effective vulnerability repair in real-world code by large language models," in *Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics*, ser. IWSPA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 23–33. [Online]. Available: https://doi.org/10.1145/3716815.3729010

[54] M. Fu, C. K. Tantithamthavorn, V. Nguyen, and T. Le, " ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We? ," in *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*. Los Alamitos, CA, USA: IEEE Computer Society, Dec. 2023, pp. 632–636. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/APSEC60848.2023.00085

[55] U. Kulsum, H. Zhu, B. Xu, and M. d'Amorim, "A case study of llm for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback," in *Proceedings of the 1st ACM International Conference on AI-Powered Software*, ser. AIware 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 103–111. [Online]. Available: https://doi.org/10.1145/3664646.3664770

[56] A. Nilizadeh, G. T. Leavens, X.-B. D. Le, C. S. Păsăreanu, and D. R. Cok, "Exploring true test overfitting in dynamic automated program repair using formal methods," in *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2021, pp. 229–240.

[57] Y. Liu, Y. Yao, J.-F. Ton, X. Zhang, R. Guo, H. Cheng, Y. Klochkov, M. F. Taufiq, and H. Li, "Trustworthy LLMs: a survey and guideline for evaluating large language models' alignment," in *Socially Responsible Language Modelling Research*, 2023. [Online]. Available: https://openreview.net/forum?id=oss9uaPFfB

[58] Y. Li, P. Branco, A. M. Hoole, M. Marwah, H. M. Koduvely, G.-V. Jourdan, and S. Jou, " SV-TrustEval-C: Evaluating Structure and Semantic Reasoning in Large Language Models for Source Code Vulnerability Analysis ," in *2025 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 3014–3032. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00191

Supplemental Material of "Benchmarking and Detecting Explanation Inconsistencies in LLM-based Vulnerability Repair"

APPENDIX A
PROMPT TEMPLATE

```
You are a security auditor. Given a vulnerability
    ↪ analysis and a patching strategy, verify
    ↪ whether the fixing action described in the
    ↪ patching strategy eliminates the
    ↪ vulnerability trigger identified in the
    ↪ vulnerability analysis.

[Few-shot Examples]

You need to follow these steps:
Step 1. Identify the vulnerability trigger in the
    ↪ vulnerability analysis.
Step 2. Identify the fixing action in the patching
    ↪  strategy.
Step 3. Verify if the action can eliminate the
    ↪ vulnerability trigger.

You should answer 'yes' or 'no'. If 'no', please
    ↪ explain why.
```

Listing 1: NLI Prompt for Assessing Logical Inconsistency

```
You are a security auditor. Please verify if the
    ↪ given explanation of fixing the
    ↪ vulnerability aligns with the patch and the
    ↪  vulnerable code.

The vulnerable code: ```[Vulnerable Code]```
The vulnerability patch: ```[Patch]```
The explanation: [Explanation]

You need to follow these steps:
Step 1. Assess the consistency between the
    ↪ explanation and the patch
Step 2. Assess the logical coherence of the
    ↪ explanation
Step 3. Assess the consistency between the
    ↪ explanation and the vulnerable code
Step 4. Answer 'yes' if all Steps 1, 2, and 3 are
    ↪ passed; otherwise, answer 'no'
```

Listing 2: Prompt of LLL-as-a-Judge Baselines

APPENDIX B
ADDITIONAL EXPERIMENTAL RESULTS

*1) **Top 25 most dangerous CWE-IDs**:* Table VIII shows the effectiiveness of SUSVF on the top 25 most dangerous CWE-IDs [42]. For CWE-125, SUSVF achieves moderate effectiveness, with Precision of 0.649, Recall of 0.661, and Specificity of 0.887. For CWE-476, CWE-787, CWE-416, and CWE-122, SUSVF demonstrates strong effectiveness, with Precision of 0.808–0.898, Recall of 0.733–0.840, and Specificity of 0.942–0.957. For the remaining, it achieves perfect scores. The results show that SusVF can maintain good effectiveness across CWE-IDs, with certain types of vulnerabilities being more difficult to detect untrustworthiness than others.

TABLE VIII: **RQ2**. The effectiveness of SUSVF across the top 25 most dangerous CWE-IDs [42]

CWE-ID	Pre	Rec	Spe
CWE-125	0.649	0.661	0.887
CWE-476	0.808	0.840	0.957
CWE-787	0.851	0.784	0.955
CWE-416	0.898	0.733	0.942
CWE-122	-	-	0.947
CWE-20	1.000	1.000	1.000
CWE-200	-	-	1.000
Others	1.000	1.000	1.000

*2) **Prompting strategies**:* The modest aggregate improvement masks a far larger change in which vulnerabilities are fixed in a trustworthy manner. Fig. 7a shows that only 203 vulnerabilities are fixed trustworthily under *both* prompting strategies, out of a union of 417. The two strategies thus agree on less than half (48.7%) of their trustworthy fixes. Although few-shot prompting overall fixes trustworthily 24 more vulnerabilities, 119 are only fixed trustworthily by few-shot prompting, and 95 are only fixed trustworthily by zero-shot prompting. In other words, few-shot prompting can trustworthily fix more vulnerabilities but also makes many fixes untrustworthy compared to zero-shot prompting. This suggests that few-shot prompting can supply reasoning scaffolding that leads to trustworthiness, but also may displace default reasoning that was better than the exemplars, resulting in more untrustworthy predictions.

Figures 7b–7f break down the results for each LLM.

- **CodeLlama** benefits the most. Under zero-shot prompting, there are only 6 out of 19 correct predictions are actually trustworthy. Few-shot prompting raises the number of trustworthy predictions from 6 to 22 (3.7×).
- **GPT-4** shows the clearest decoupling of correctness and trustworthiness. Few-shot prompting adds a single correct patch (91 → 92) but yields 7 additional trustworthy predictions.
- **Claude-O 4.1** is the only backend that degrades on both axes under few-shot prompting: 8 fewer correct patches and 10 less trustworthy ones. It also exhibits the highest overlap between two prompting strategies. This indicates that few-shot prompting adds limited gain to stronger LLMs. It also occasionally displaces reasoning that was better than the exemplars, resulting in more untrustworthy predictions.
- **Qwen-Coder** gains the most correct patches (+13) and also 10 more trustworthy ones.
- **DS-Coder** achieves similar total numbers of correct and trustworthy predictions under both prompting strategies.

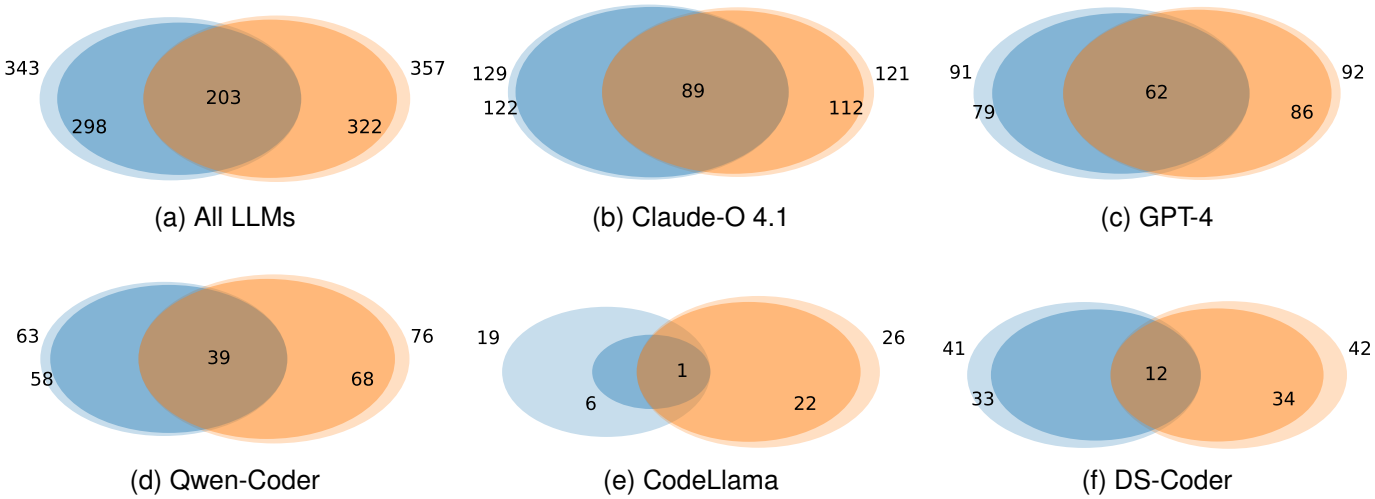


Fig. 7: Comparing zero-shot prompting and few-shot prompting. Lighter and darker shades represent correct and trustworthy predictions, respectively. The number at the centre indicates the number of vulnerabilities fixed by trustworthy predictions in both prompting strategies.

But similar to other LLMs, among the trustworthy cases, only 12 are both trustworthy, while 21 are only trustworthy under zero-shot prompting and 22 are only trustworthy under few-shot prompting.

The per-backend results suggest that the benefit of few-shot prompting may be capability-dependent: the LLMs with the weakest zero-shot reasoning gain the most, while the LLMs with the strongest zero-shot reasoning can deteriorate. This is reflected in the dispersion of the trustworthy yield across backends. A plausible explanation is that exemplars provide a template that links the vulnerability analysis to the fixing action, which suppresses logical inconsistencies within the explanation for weaker models, but can also bias the explanation towards the retrieved exemplar's fixing action when the model would otherwise have reasoned correctly on its own.

*3) **Single-hunk vs. multi-hunk vulnerabilities**:* Table IX shows that LLM-based AVR systems generate fewer correct patches for multi-hunk vulnerabilities (538) compared to those for single-hunk vulnerabilities (1140). This means that multi-hunk vulnerabilities are naturally more difficult to fix, as they span multiple locations. Among correct patch predictions, the proportion of untrustworthy predictions is higher for single-hunk vulnerabilities (18.9%) than for multi-hunk ones (13.2%). This suggests that, for single-hunk vulnerabilities, LLMs are more likely to rely on spurious reasons to generate correct patches. In contrast, multi-hunk vulnerabilities typically require coordinating multiple, interprocedural changes, making it harder for LLMs to use superficial or spurious reasoning. As a result, there are fewer untrustworthy predictions for fixing multi-hunk vulnerabilities.

SUSVF performs consistently well across both single-hunk and multi-hunk vulnerabilities, although it is noticeably more effective on single-hunk vulnerabilities. In this case, SUSVF achieves high Precision of 0.886 and strong Recall of 0.755.

TABLE IX: **RQ2**. SUSVF's effectiveness on single-hunk and multi-hunk vulnerabilities

	Correct	Untr.	Pre	Rec	Spe	MCC
Single-Hunk	1140	216 (18.9%)	0.886	0.755	0.977	0.780
Multi-Hunk	538	71 (13.2%)	0.870	0.662	0.985	0.729

This indicates that SUSVF not only identifies most untrustworthy predictions but also keeps FPs relatively low. This is further supported by the very high Specificity of 0.977, demonstrating that trustworthy predictions are rarely misclassified. The overall balance metric is reflected in MCC of 0.780, suggesting SUSVF's effectiveness when handling predictions for fixing vulnerabilities within a single code region.

SUSVF's effectiveness slightly drops for multi-hunk vulnerabilities, which are inherently more complex. While Precision and Specificity remain relatively high at 0.870 and 0.985, Recall and MCC decrease to 0.662 and 0.729, by 12% and 7%, respectively. This indicates that it is easier to detect untrustworthy predictions for single-hunk vulnerabilities.

*4) **Stochastic mature of trustworthiness**:* For each vulnerability that Vul-R2 fixes correctly in at least one run, we compute its trustworthiness rate: the fraction of its 10 runs that produce a trustworthy prediction, i.e., a correct patch with a correct explanation. A rate of 1.0 means every run is trustworthy. A rate of 0.0 means Vul-R2 produced correct patches for the vulnerability, but never with a correct explanation. Figure 2 shows the distribution of this rate on each dataset.

Figure 2 shows that trustworthiness is highly unstable across runs:

- Most runs are not trustworthy, even for vulnerabilities Vul-R2 can fix correctly. The median vulnerability receives a trustworthy prediction in only 3 of 10 runs on PrimeVul and 2 of 10 runs on SVEN. A random single

TABLE X: **RQ2**. SUSVF's effectiveness on different LLM-based AVR systems

System Under Evaluation	Correct Patches	Untrustworthy	Pre	Rec	Spe	MCC
Vul-R2	978	207 (21.2%)	0.882	0.758	0.973	0.774
IP w/ GPT-4	91	12 (13.2%)	0.909	0.833	0.987	0.852
APPATCH w/ GPT-4	92	6 (6.5%)	0.800	0.667	0.988	0.713
IP w/ Claude-O 4.1	129	7 (5.4%)	0.750	0.429	0.992	0.549
APPATCH w/ Claude-O 4.1	121	9 (7.4%)	0.857	0.667	0.991	0.739
IP w/ Qwen-Coder	63	5 (7.9%)	0.600	0.600	0.966	0.566
APPATCH w/ Qwen-Coder	76	8 (10.5%)	0.800	0.500	0.985	0.600
IP w/ CodeLlama	19	13 (68.4%)	1.000	0.846	1.000	0.797
APPATCH w/ CodeLlama	26	4 (15.4%)	1.000	0.500	1.000	0.677
IP w/ DS-Coder	41	8 (19.5%)	1.000	0.625	1.000	0.757
APPATCH w/ DS-Coder	42	8 (19.1%)	1.000	0.625	1.000	0.758

: the best value in each column, : the worst value in each column

run is therefore more likely to be untrustworthy.

- The same vulnerability is fixed trustworthily in some runs and not in others. The interquartile ranges span roughly 0.1 to 0.6 on PrimeVul and 0.1 to 0.5 on SVEN. For most vulnerabilities, whether Vul-R2's output can be trusted depends on which run one happens to sample.
- Consistently trustworthy repair is rare. On SVEN, no vulnerability is fixed trustworthily in all 10 runs (maximum 0.9). At the other extreme, some vulnerabilities on both datasets have a rate of 0: Vul-R2 fixes them correctly in some runs, but never with a correct explanation.
- SVEN is slightly harder than PrimeVul for Vul-R2. Its median rate is lower (0.2 vs. 0.3), consistent with its higher share of untrustworthy correct patches (24.9% vs. 18.4%, Table II).

Trustworthiness across vulnerability types. Table XI reports, for each CWE-ID, the trustworthiness rate over its vulnerabilities.

We observed three key results as follows:

- Stability varies strongly across vulnerability types. Mean rates range from 0.0% (CWE-400) to 95.0% (CWE-193). Only 10 of the 30 CWE types reach 50%, and 10 are below 25%.
- The most CWEs are fixed trustworthily in only about one run in three. The four most frequent CWE types in VFXplainBench (Table IV), CWE-125, CWE-476, CWE-787 and CWE-416, have mean rates between 29.6% and 37.6%.
- Vulnerabilities of the same type can behave very differently. Some types show large spread, such as CWE-20 (50.0% ± 40.0%), CWE-122 (55.0% ± 45.0%) and CWE-617 (30.0% ± 32.5%). The CWE type alone does not predict whether repair will be trustworthy.

5) ***SUSVF's effectiveness on different LLM-based AVR systems:*** Table X evaluates how SUSVF performs across different LLM-based AVR systems. We evaluated 11 system settings: Vul-R2 and 10 combinations of five LLM backends with two approaches (IP and ApPatch).

SUSVF consistently achieves high Precision and Specificity across almost all systems, often exceeding 0.800 and in several cases reaching 1. This indicates that the method is highly precise in flagging untrustworthy predictions, producing very few FPs. In practice, this means that predictions detected by SUSVF are very likely untrustworthy, which is particularly desirable in security-sensitive AVR settings where untrustworthy predictions could mislead developers or disrupt workflows.

In contrast, Recall varies substantially across systems. Some settings, such as IP w/ GPT-4 and CodeLlama, achieve strong Recall (above 0.833). The others, notably IP w/ Claude-O 4.1, drop to much lower levels (0.429). As we acknowledge, SUSVF cannot exhaustively detect all untrustworthy predictions, leading to FNs.

MCC provides a balanced view of effectiveness. SUSVF performs best on IP w/ GPT-4, achieving the highest MCC of 0.852, and maintains strong results on most other systems. However, its MCC declines for IP w/ Claude-O 4.1 and Qwen-Coder, where MCC values fall closer to 0.55–0.60. We argue that low effectiveness in some cases is not due to SUSVF's limitation, but rather because there are only a few untrustworthy examples (4–5), such as IP w/ Qwen-Coder and APPATCH w/ CodeLlama. Overall, these results demonstrate that SUSVF is a reliable high-precision detector of untrustworthy predictions, making it well suited for deployment in security-critical scenarios.

6) ***Sensitivity analysis:*** The detailed results are shown in Figure 8. Compared with LLM-as-a-Judge baselines, SUSVF statistically significantly outperforms them across all NLI backends. Specifically, it achieves 52%–589% higher Precision, 0%–4,553% higher Recall, and 93%–4,367% higher MCC. Improvements in Specificity are more modest (up to 30%). Notably, SUSVF with Qwen2.5-Coder-32B-Instruct also outperforms its corresponding Judge baseline.

Among the NLI machines, GPT-4o-mini, Claude-Haiku-4.1, and Qwen2.5-Coder-32B-Instruct achieve the highest effectiveness, with Precision of 0.737–0.882, Recall of 0.732–0.791, Specificity of 0.942–0.980, and MCC of 0.711–0.768. The other LLMs show relatively lower effectiveness, with Precision of 0.611–0.765, Recall of 0.544–0.707, Specificity of 0.907–0.966, and MCC of 0.581–0.610. While Qwen2.5-Coder-32B-Instruct demonstrates the best effectiveness among the Qwen2.5-Coder-x-Instruct LLMs, we observe no big differences between the three smaller respective LLMs.

TABLE XI: Trustworthiness across vulnerability types

CWE-ID	Trustworthiness	CWE-ID	Trustworthiness	CWE-ID	Trustworthiness
CWE-17	80.0% ± 0.0%	CWE-193	95.0% ± 5.0%	CWE-416	37.6% ± 11.2%
CWE-20	50.0% ± 40.0%	CWE-200	30.0% ± 20.0%	CWE-476	35.9% ± 9.5%
CWE-59	90.0% ± 0.0%	CWE-276	50.0% ± 0.0%	CWE-617	30.0% ± 32.5%
CWE-119	20.0% ± 30.0%	CWE-284	10.0% ± 10.0%	CWE-665	10.0% ± 0.0%
CWE-120	5.0% ± 5.0%	CWE-288	20.0% ± 0.0%	CWE-703	31.0% ± 22.0%
CWE-122	55.0% ± 45.0%	CWE-362	20.0% ± 20.0%	CWE-772	10.0% ± 0.0%
CWE-125	29.6% ± 7.2%	CWE-369	62.0% ± 36.0%	CWE-787	32.9% ± 8.8%
CWE-134	60.0% ± 0.0%	CWE-400	0.0% ± 0.0%	CWE-835	43.3% ± 15.0%
CWE-189	25.0% ± 5.0%	CWE-401	83.3% ± 20.0%	CWE-843	30.0% ± 0.0%
CWE-190	23.8% ± 8.1%	CWE-415	56.0% ± 34.0%	CWE-862	10.0% ± 0.0%

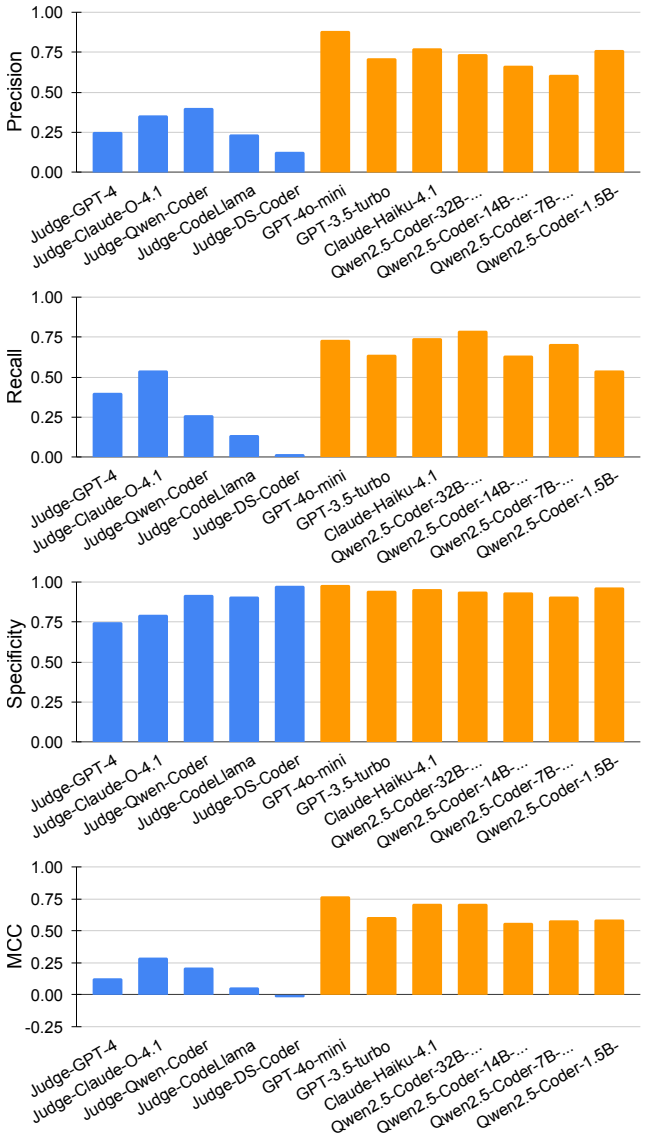


Fig. 8: **RQ5**. The effectiveness of LLM-as-a-Judge and SusVF (ours) with different LLMs as the NLI machine.

APPENDIX C
CASE STUDY

False positive: We provide an example of FPs. The function `decode_dds1` contains the CVE-2017-9992[8] vulnerability. Specifically, the boundary check `if (frame_end - frame < width + 3)` at Line 35 is off by 1, causing a heap-based buffer overflow when writing data to `frame` at Lines 37–42.

```
1  static int decode_dds1(GetByteContext *gb,
       uint8_t *frame, int width, int height)
2  {
3      const uint8_t *frame_start = frame;
4      const uint8_t *frame_end   = frame + width *
       height;
5      int mask = 0x10000, bitbuf = 0;
6      int i, v, offset, count, segments;
7
8      segments = bytestream2_get_le16(gb);
9      while (segments--) {
10         if (bytestream2_get_bytes_left(gb) < 2)
11             return AVERROR_INVALIDDATA;
12         if (mask == 0x10000) {
13             bitbuf = bytestream2_get_le16u(gb);
14             mask = 1;
15         }
16
17         if (bitbuf & mask) {
18             v = bytestream2_get_le16(gb);
19             offset = (v & 0x1FFF) << 2;
20             count = ((v >> 13) + 2) << 1;
21             if (frame - frame_start < offset ||
       frame_end - frame < count*2 + width)
22                 return AVERROR_INVALIDDATA;
23             for (i = 0; i < count; i++) {
24                 frame[0] = frame[1] =
25                 frame[width] = frame[width + 1] =
        frame[-offset];
26
27                 frame += 2;
28             }
29         } else if (bitbuf & (mask << 1)) {
30             v = bytestream2_get_le16(gb)*2;
31             if (frame - frame_end < v)
32                 return AVERROR_INVALIDDATA;
33             frame += v;
34         } else {
35             if (frame_end - frame < width + 3)
36                 return AVERROR_INVALIDDATA;
37             frame[0] = frame[1] =
38             frame[width] = frame[width + 1] =
       bytestream2_get_byte(gb);
39             frame += 2;
40             frame[0] = frame[1] =
41             frame[width] = frame[width + 1] =
       bytestream2_get_byte(gb);
42             frame += 2;
43         }
44         mask <<= 2;
45     }
46
47     return 0;
48 }
```

APPATCH w/ GPT-4 makes a prediction for fixing this vulnerable code as follows.

```
Vulnerability analysis:
At line 4, "frame_end" is calculated as the end of
    ↪ the frame buffer.
```

[8]https://nvd.nist.gov/vuln/detail/CVE-2017-9992

```
At lines 21-22 and 35-36, there are checks to
    ↪ ensure that "frame" does not go out of
    ↪ bounds.
At line 40, "frame" is written with an offset of "
    ↪ width + 1". However, the check at line 35
    ↪ only ensures that "frame_end - frame" is at
    ↪  least "width + 3", which may not be
    ↪ sufficient to prevent an out-of-bound write
    ↪  when "frame" is incremented by 2 at line
    ↪ 42. This can cause a CWE-787 out-of-bound
    ↪ write vulnerability.

Fixing action:
To avoid the out-of-bound write (CWE-787)
    ↪ vulnerability at line 40, we should ensure
    ↪ that "frame_end - frame" is sufficient to
    ↪ accommodate the writes at lines 40-42.

Patch:
```
- if (frame_end - frame < width + 3)
+ if (frame_end - frame < width + 4)
```
```

The generated patch is correct, and the explanation is also correct, making this prediction trustworthy. SUSVF applies Gumtree to extract differences between the vulnerable and patched code. The Gumtree's output is as follows.

```
[{
    "action": "update-node",
    "new_tree": "4",
    "old_tree": "3",
    "code": "width + 4"
    "parent": "if (frame_end - frame < width + 4)
    ↪ return AVERROR_INVALIDDATA;"
}]
```

By summarizing these differences, SUSVF produces the change description "*Updating the condition to check if the difference between frame_end and frame is less than width + 4 instead of width + 3 before returning the error code AVERROR_INVALIDDATA.*" This description correctly describes the changes needed to fix the vulnerability. However, the assessment of semantic equivalence in Stage ① struggles to compare this change description and the explanation. As a result, SUSVF thinks they are not equivalent, misclassifying the prediction as untrustworthy.

In this case, the fixing action states the intent of the change ("ensure that `frame_end - frame` is sufficient to accommodate the writes at Lines 40–42"), whereas the change description states the concrete edit (`width + 3` to `width + 4`). The two differ in level of abstraction, not in meaning, so the explanation is consistent with the patch. This false positive illustrates the residual measurement error discussed in Section VII.

False negative 1: We provide an example of FPs. The function `nfc_genl_deactivate_target` contains the CVE-2019-12984[9] vulnerability. Specifically, a NULL pointer dereference vulnerability can be triggered at Line 17 if the attribute `NFC_ATTR_TARGET_INDEX` is omitted. However, there is no condition check for this.

```
1 static int nfc_genl_deactivate_target(struct
      sk_buff *skb,
2                 struct genl_info *info)
3 {
4   struct nfc_dev *dev;
5   u32 device_idx, target_idx;
6   int rc;
7
8   if (!info->attrs[NFC_ATTR_DEVICE_INDEX])
9     return -EINVAL;
10
11  device_idx = nla_get_u32(info->attrs[
      NFC_ATTR_DEVICE_INDEX]);
12
13  dev = nfc_get_device(device_idx);
14  if (!dev)
15    return -ENODEV;
16
17  target_idx = nla_get_u32(info->attrs[
      NFC_ATTR_TARGET_INDEX]);
18
19  rc = nfc_deactivate_target(dev, target_idx,
      NFC_TARGET_MODE_SLEEP);
20
21  nfc_put_device(dev);
22  return rc;
23 }
```

Vul-R2 makes a prediction for fixing this vulnerability as follows.

```
Vulnerability analysis:
1. Locate the dereference
   nla_get_u32(info->attrs[NFC_ATTR_DEVICE_INDEX])
    ↪  is executed immediately after the
    ↪ attribute is tested.

2. Faulty test
   if (!info->attrs[NFC_ATTR_DEVICE_INDEX])
    ↪ returns true when the pointer is NULL, but
    ↪ the code only rejects the message with -
    ↪ EINVAL.
   The same check is done again (inside
    ↪ nla_get_u32) and, if the pointer is still
    ↪ NULL, the kernel macro dereferences a NULL
    ↪ pointer, generating an oops / BUG.

3. Reproducer
   - Open a Generic-Netlink connection to the ``
    ↪ nfc'' family.
   - Send a command that omits the mandatory
    ↪ NFC_ATTR_DEVICE_INDEX attribute while
    ↪ setting TARGET_INDEX.
   => First check passes, pointer is NULL, second
    ↪ check in nla_get_u32 fires, kernel crashes.

4. Root cause
   Missing defensive check for the DEVICE_INDEX
    ↪ attribute allows a NULL pointer to flow
    ↪ into nla_get_u32, which then dereferences
    ↪ it, triggering a NULL-pointer dereference
    ↪ in kernel space.

Fixing action:
We should add a second level NULL test and bail
    ↪ out early. With both attributes validated
    ↪ first, no NULL pointer reaches nla_get_u32
    ↪ and the vulnerability is removed.

The patch should be:
```
    if (!info->attrs[NFC_ATTR_DEVICE_INDEX] ||
        !info->attrs[NFC_ATTR_TARGET_INDEX])
            return -EINVAL;
```
```

The generated patch is correct. However, the explanation is also logically inconsistent. It identifies the root cause of the vulnerability as missing a check for the DEVICE_INDEX attribute, while the fixing action suggests adding a check for the TARGET_INDEX attribute. The fixing action is correct,

[9]https://nvd.nist.gov/vuln/detail/CVE-2019-12984

but the vulnerability analysis is not. As a result, this prediction of Vul-R2 is untrustworthy.

In Stage ②, SUSVF's NLI machine hallucinates, identifying that the fixing action and the vulnerability analysis are logically consistent. As a result, SUSVF misclassifies the prediction as trustworthy.

False negative 2: A few cases of FNs are due to the awkwardness or ambiguity of the explanations. For example, the function `pci_read` contains the CVE-2014-5388[10] vulnerability. It may out-of-bounds assess the array `acpi_pcihp_pci_status` in the switch-case statement at Line 10–39, as the boundary check at Line 6 is incorrect.

```
1  static uint64_t pci_read(void *opaque, hwaddr
       addr, unsigned int size) {
2      AcpiPciHpState *s = opaque;
3      uint32_t val = 0;
4      int bsel = s->hotplug_select;
5
6      if (bsel < 0 || bsel >
       ACPI_PCIHP_MAX_HOTPLUG_BUS) {
7          return 0;
8      }
9
10     switch (addr) {
11         case PCI_UP_BASE:
12             val = s->acpi_pcihp_pci_status[bsel].
       up;
13             if (!s->legacy_piix) {
14                 s->acpi_pcihp_pci_status[bsel].up
        = 0;
15             }
16             ACPI_PCIHP_DPRINTF("pci_up_read %"
       PRIu32 "\n", val);
17             break;
18
19         case PCI_DOWN_BASE:
20             val = s->acpi_pcihp_pci_status[bsel].
       down;
21             ACPI_PCIHP_DPRINTF("pci_down_read %"
       PRIu32 "\n", val);
22             break;
23
24         case PCI_EJ_BASE:
25             ACPI_PCIHP_DPRINTF("pci_features_read
        %" PRIu32 "\n", val);
26             break;
27
28         case PCI_RMV_BASE:
29             val = s->acpi_pcihp_pci_status[bsel].
       hotplug_enable;
30             ACPI_PCIHP_DPRINTF("pci_rmv_read %"
       PRIu32 "\n", val);
31             break;
32
33         case PCI_SEL_BASE:
34             val = s->hotplug_select;
35             ACPI_PCIHP_DPRINTF("pci_sel_read %"
       PRIu32 "\n", val);
36
37         default:
38             break;
39     }
40
41     return val;
42 }
```

APPATCH w/ DeepSeek-Coder generates a correct patch, with an explanation seemingly convincing.

```
Vulnerability analysis:
At line 5, "bsel" is calculated from "s->
    ↪ hotplug_select".
At line 13, "bsel" is used to index "s->
    ↪ acpi_pcihp_pci_status".
However, if "bsel" is less than 0 or greater than
    ↪ "ACPI_PCIHP_MAX_HOTPLUG_BUS", it will cause
    ↪  an out-of-bound access (CWE-787)
    ↪ vulnerability.

Patching strategy:
To avoid the out-of-bound access (CWE-787)
    ↪ vulnerability at line 13, we should ensure
    ↪ that "bsel" is within the valid range,
    ↪ which can be done by adding a condition at
    ↪ line 6.

Therefore, the patch should be:
```
- if (bsel < 0 || bsel >
    ↪ ACPI_PCIHP_MAX_HOTPLUG_BUS)
+ if (bsel < 0 || bsel >=
    ↪ ACPI_PCIHP_MAX_HOTPLUG_BUS)
```

The fixing action "*ensure that bsel is within the valid range*" is consistent with the patch, despite being described in a quite general manner. The fixing action is also logically consistent with the vulnerability analysis, the vulnerability occurs "*if bsel is less than 0 or greater than ACPI_PCIHP_MAX_HOTPLUG_BUS*". However, this vulnerability analysis is actually incorrect, as the vulnerability also occurs if bsel is equal to ACPI_PCIHP_MAX_HOTPLUG_BUS. Therefore, this prediction is untrustworthy. SUSVF fails to detect this subtle untrustworthiness, misclassifying this prediction as trustworthy.

## APPENDIX D
## MANUAL LABELING

***Partially correct patches:*** Partially correct patches are labeled incorrect. We observed a few cases in which the patch does not sufficiently address the vulnerability. The following is one of them. A program `mongo_data_append` incorrectly uses `int` rather than `size_t` for the parameter `len`. This could cause an integer overflow at Line 2 when calling the function `memcpy`.

```
1 static char *mongo_data_append(char *start, const
      void *data, int len) {
2     memcpy(start, data, len);
3     return start + len;
4 }
```

CVE-2020-12135

Claude-Opus 4.1 generated a patch that adds a conditional check `if (len < 0)` at the beginning of the function. This if statement can avoid the case when `len` is negative. However, `len` remains an int, so the `int`/`size_t` mismatch behind the vulnerability is not fixed: a large unsigned length can still be truncated on its way to `memcpy`. In this case, the generated patch only partially addresses the vulnerability.

```
1 static char *mongo_data_append(char *start, const
      void *data, int len) {
2     if (len < 0)
3         return start;
4     memcpy(start, data, len);
5     return start + len;
6 }
```

A patch generated by Claude-Opus 4.1

We label such partially correct patches as incorrect. Patches on which the annotators were unsure were discussed with the third annotator to determine their correctness.

[10]https://nvd.nist.gov/vuln/detail/CVE-2014-5388
```

1. Buggy Code Excluded for brevity

2. Bug Info

```
# Vulnerability Description

FFmpeg before 2017-02-07 has an out-of-bounds write caused by a heap-based buffer overflow related to the
decode_frame function in libavcodec/pictordec.c.

# Fixing Commit
URL: https://github.com/FFmpeg/FFmpeg/commit/8c2ea3030af7b40a3c4275696fb5c76cdb80950a

avcodec/pictordec: Fix logic error
Fixes: 559/clusterfuzz-testcase-6424225917173760

Found-by: continuous fuzzing process https://github.com/google/oss-fuzz/tree/master/targets/ffmpeg
Signed-off-by: Michael Niedermayer <michael@niedermayer.cc>
```

3. Developer Fix

libavcodec/pictordec.c CHANGED

```
@@ -142,7 +142,7 @@ static int decode_frame(AVCodecContext *avctx,
142 142
143 143       if (av_image_check_size(s->width, s->height, 0, avctx) < 0)
144 144           return -1;
145     -     if (s->width != avctx->width && s->height != avctx->height) {
    145 +     if (s->width != avctx->width || s->height != avctx->height) {
146 146           ret = ff_set_dimensions(avctx, s->width, s->height);
147 147           if (ret < 0)
148 148               return ret;
```

4. LLM Fix

Buggy → Fixed RENAMED

```
@@ -42,9 +42,9 @@
42 42       avctx->pix_fmt = AV_PIX_FMT_PAL8;
43 43
44 44       if (av_image_check_size(s->width, s->height, 0, avctx) < 0)
45 45           return -1;
46     -     if (s->width != avctx->width && s->height != avctx->height) {
   46 +     if (s->width != avctx->width || s->height != avctx->height) {
47 47           ret = ff_set_dimensions(avctx, s->width, s->height);
48 48           if (ret < 0)
49 49               return ret;
50 50       }
```

5. LLM Explanation

Summary of analysis (abridged):

1. Trigger & symptom
   • Feed a packet that starts with width/height fields different from the codec context's current dimensions.
   • The original guard only re-initialises dimensions when both width and height differ, so a packet with a new width but same height (or vice-versa) slips through.
   • `ff_set_dimensions()` allocates/initialises internal buffers based on the new size. Subsequent image-fill routines (`picmemset_*`) still use the old `width * height` product when writing, resulting in writes past the end of the newly allocated picture buffer — an Out-of-Bounds Write (CWE-787).

2. Root cause
   The equality test guarding the re-initialisation call is incomplete; only "both width and height differ" is checked. If only one dimension changes, the code skips re-initialisation, keeps stale buffer sizes, and proceeds to decode, producing OOB writes.

3. Fix
   Extend the guard to cover any dimension change:
   ```c
   // old
   if (s->width != avctx->width && s->height != avctx->height) {

   // new
   if (s->width != avctx->width || s->height != avctx->height) {
   ```
   This ensures that any size mismatch leads to a fresh `ff_set_dimensions()` call, guaranteeing that internal buffers match the picture that is about to be decoded and eliminating the Out-of-Bounds Write.

Fig. 9: Information provided to annotators for labeling.